Bachelor of Business Information Technology

PRACTICAL LOGBOOK

BIT3208: Advanced Web Design and Development

Weeks 1–8 — ShopMart E-Commerce Platform, Student Portal Login System & Responsive Web Design Practicals

Student Name	Stephen Otiende
Admission Number	BBIT/2018/36596
Course / Class	Bachelor of Business Information Technology
Unit Code	BIT3208
Unit Name	Advanced Web Design and Development
Lecturer Name	Mr. Michael Nyoro
Semester / Year	Semester 2 / 2025–2026
GitHub / Portfolio	https://github.com/Otyzo/Advanced-web-design-Tasks

Table of Contents

Part A — ShopMart Practical Logbook (Weeks 1–5)

A PHP/MySQL e-commerce platform developed incrementally across Weeks 1 to 5, progressing from local environment setup and interface planning through client-side validation, server-side authentication, and full CRUD database functionality.

Week 1: Local Environment Setup

Installation and Testing of Local Development Environment

Title

Installation and Testing of Local Development Environment

What Was Built

Before writing a single line of ShopMart's application code, I installed and rigorously verified the local development stack using XAMPP, which bundles Apache, MySQL, and PHP into a single, version-matched environment. Rather than assuming the installation had succeeded, I wrote three small diagnostic scripts to isolate and confirm each layer independently: a Hello World script to prove Apache was correctly invoking the PHP interpreter, a mysqli connection script to confirm PHP could authenticate against MySQL at the driver level, and a JSON-returning endpoint that I exercised through Postman to inspect the raw HTTP method, headers, and body PHP receives on a request. Testing each layer in isolation before combining them meant that any later fault in ShopMart proper could be traced to application logic rather than to an unverified environment.

Required Evidence

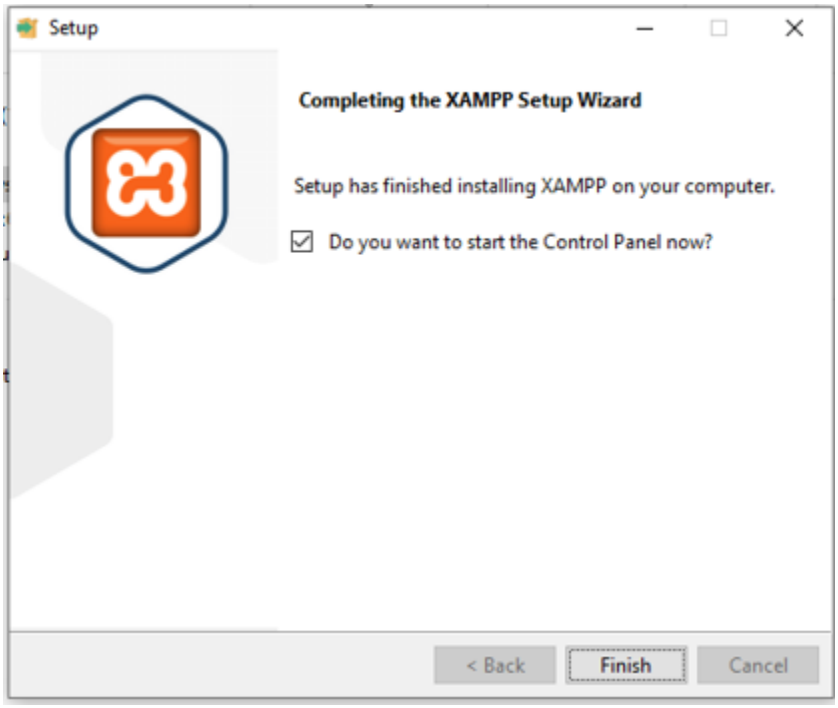


Fig 1: XAMPP Installation Complete — The XAMPP setup wizard confirming successful installation on Windows, with the option to launch the Control Panel.

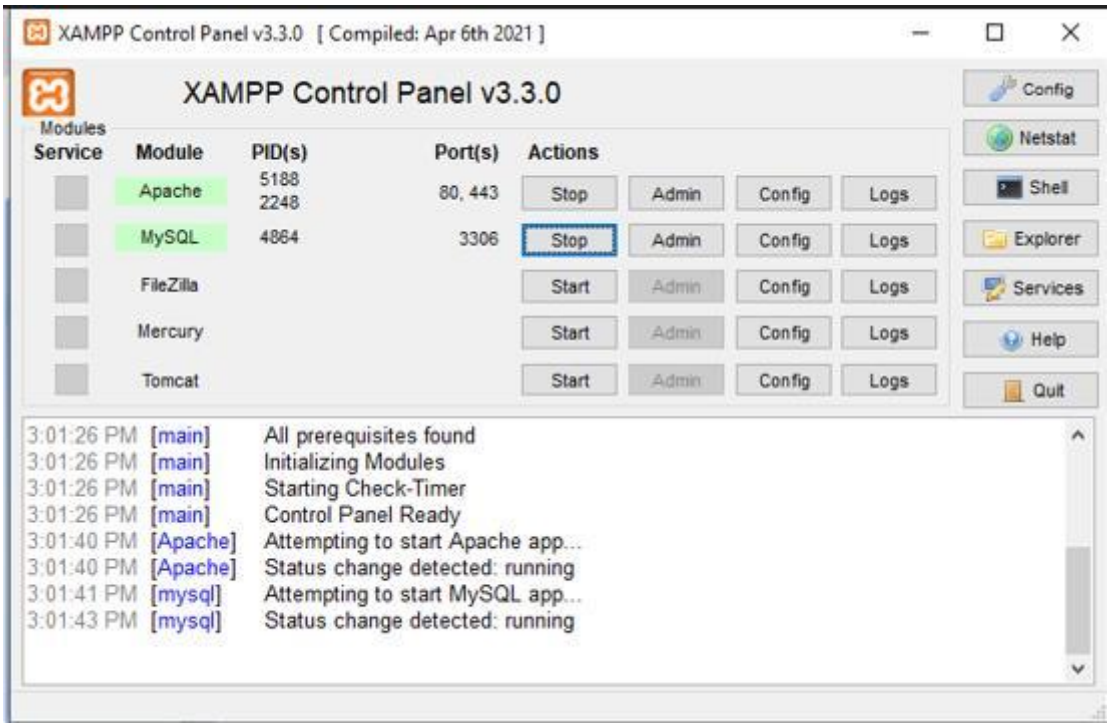


Fig 2: Apache and MySQL Running — XAMPP Control Panel showing both Apache and MySQL with a green "Running" status.

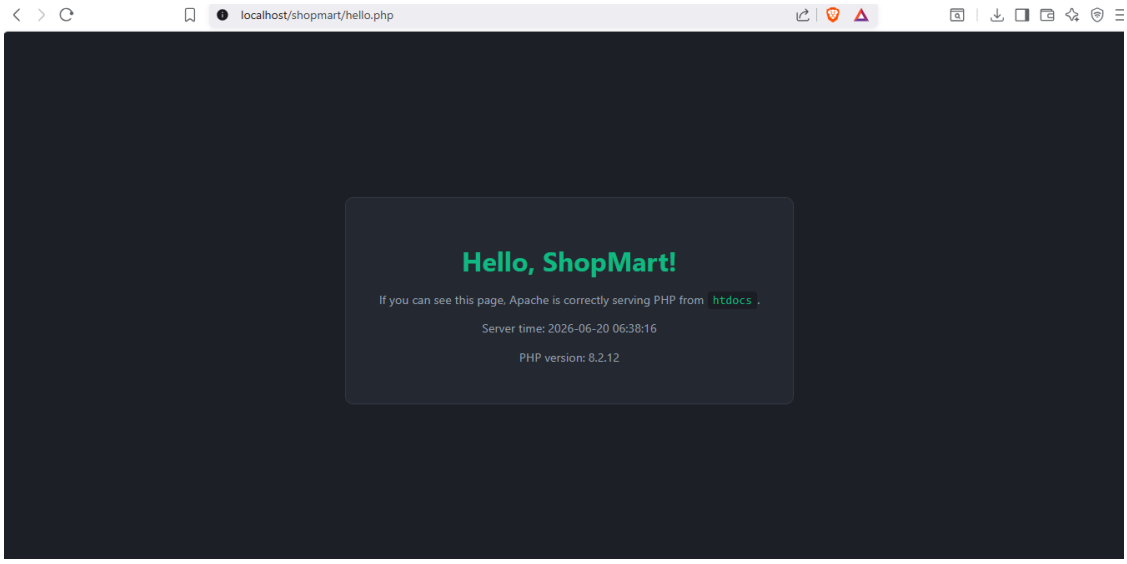


Fig 3: Hello World PHP Test — Browser showing hello.php with the live server time and PHP version, confirming PHP is executing correctly.

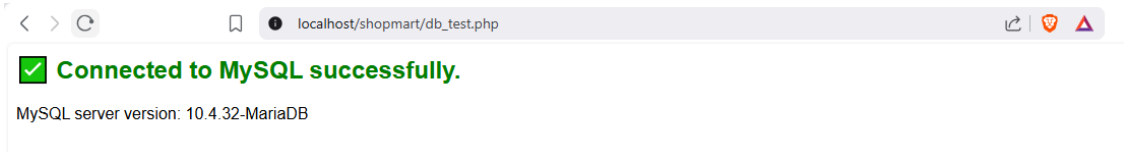


Fig 4: Database Connection Test — db_test.php showing a successful MySQL connection message.

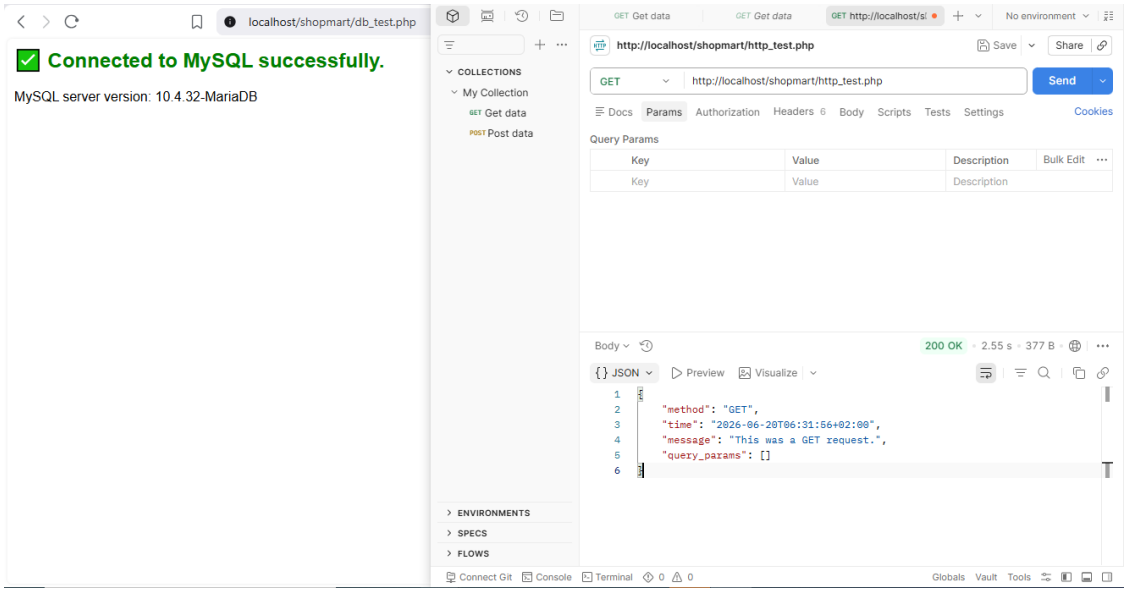


Fig 5: HTTP Test in Postman — A Postman GET request to http_test.php returning the expected JSON response.

Student Reflection

> *This week was less about producing a feature and more about establishing a reliable foundation to build on. Seeing hello.php return the correct server time and PHP version confirmed that Apache was delegating requests to the PHP interpreter as expected, and db_test.php's successful mysqli handshake confirmed the database layer was reachable independently of any application logic. Inspecting http_test.php through Postman was the most valuable exercise of the three: watching the exact method, headers, and body PHP receives on a request turned the request–response cycle from a textbook abstraction into something I could observe directly. The significance for ShopMart is structural rather than cosmetic — every feature I build from this point forward, whether it is form submission, authentication, or product management, depends on this same Apache–PHP–MySQL chain holding up, so verifying it early removes an entire class of failure from later debugging.*

Tools Used

- XAMPP — local Apache web server, MySQL database, and PHP runtime
- Postman — sending GET/POST requests and inspecting responses
- Web Browser — verifying localhost and viewing PHP output

Week 2: Wireframes and GUI Design

User Interface Planning and System Design

Title

User Interface Planning and System Design

What Was Built

Before committing to any backend implementation, I deliberately separated the interface design process into two distinct stages, reflecting standard UI/UX practice. First, I produced low-fidelity wireframes for the four core screens — Login, Dashboard, Homepage, and Product page — using the conventional greyscale box-and-label notation. Keeping these wireframes free of colour and typography was intentional: it forced every layout decision to be justified on structure and information hierarchy alone, rather than being influenced by visual polish. Once the structural layout was validated, I progressed to a higher-fidelity GUI concept that defined ShopMart's actual visual identity — a dark charcoal background with emerald green as the singular accent colour, applied consistently across buttons, links, and highlights to establish a coherent design language across every page.

Required Evidence

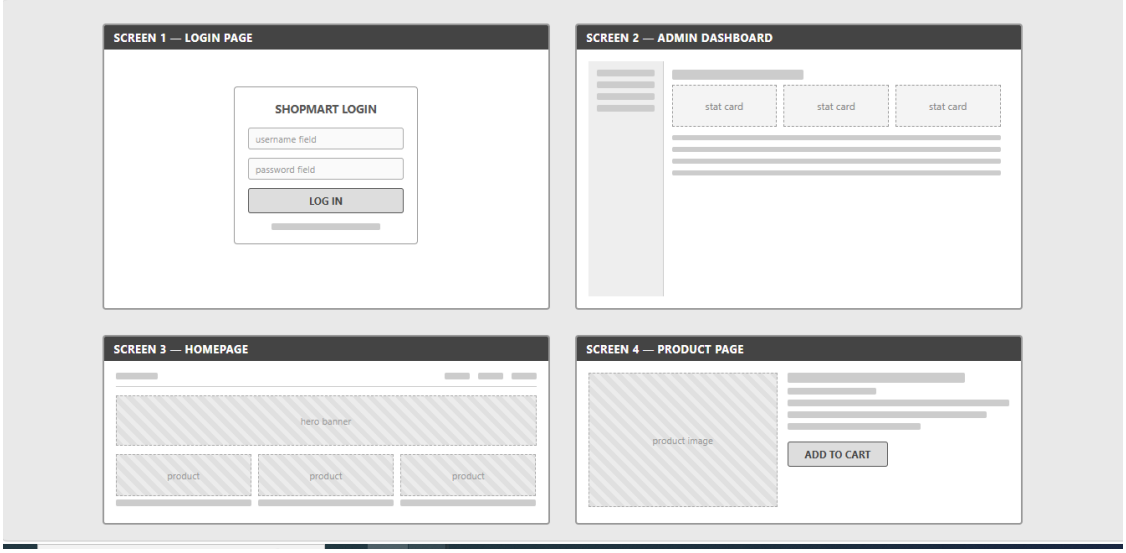


Fig 1: Wireframes — Login, Dashboard, Homepage, Product Page — Low-fidelity wireframe layouts showing the four core ShopMart screens in greyscale box-and-label form.

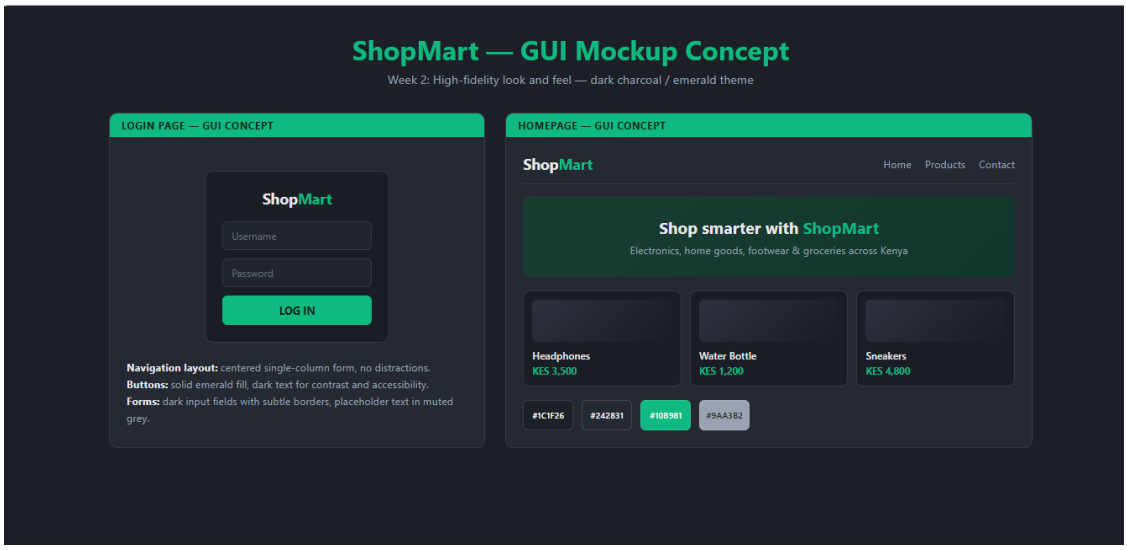


Fig 2: GUI Concept — Color Palette and Login Mockup — Higher-fidelity mockup showing the dark charcoal / emerald green color scheme applied to the Login screen.

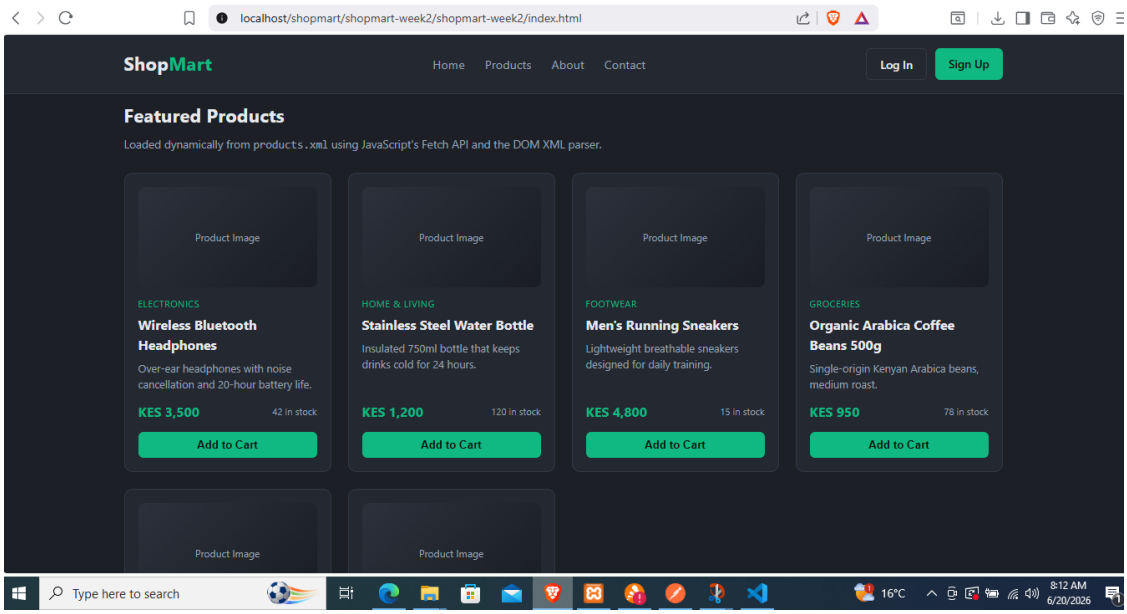


Fig 3: GUI Concept — Homepage Mockup — Higher-fidelity mockup showing the homepage hero section and product card layout in the final color scheme.

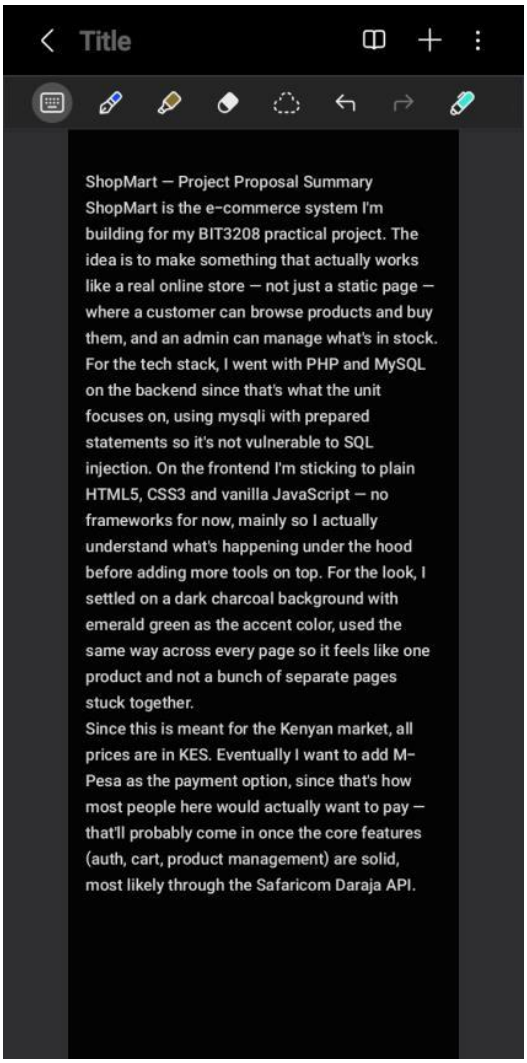


Fig 4: Project Proposal Summary — Written summary outlining ShopMart's purpose, chosen technologies (PHP, MySQL, HTML5/CSS3/JS), and planned M-Pesa integration.

Fig 5: Folder Structure Planning — Planned folder structure showing one directory per week (Week1–Week5) plus a Documentation folder, per the version-control guidance for this unit.

Student Reflection

> *Sequencing the design phase ahead of any backend work paid off noticeably in later weeks. Sketching the Dashboard wireframe first forced me to think through exactly what data and controls the admin side actually needed before I had a database to populate it with, and settling on the emerald-on-charcoal palette early meant the visual identity never had to be retrofitted once functional pages existed. The main discipline I had to enforce on myself was resisting the urge to introduce colour before the structural layout was confirmed — wireframing is only useful if it stays a layout exercise rather than becoming a half-finished mockup. For ShopMart specifically, this matters because a validated plan converts UI decisions from something I would otherwise be making ad hoc while writing PHP into something already settled, which removes a source of rework once the real backend logic and database schema are in place.*

Tools Used

- HTML/CSS (in place of Figma) — used to produce both the wireframes and the GUI mockup
- Browser — viewing and screenshotting both deliverables

Week 3: JavaScript and PHP Basics

Frontend Interaction and Backend Foundations

Title

Client-Side Form Validation for ShopMart Registration Using JavaScript

What Was Built

I built a standalone ShopMart registration form (username, email, password, confirm password) in HTML5, CSS3, and vanilla JavaScript, with validation handled entirely client-side at this stage. The form intentionally does not yet talk to PHP or MySQL, since the brief scheduled backend integration for later weeks — the objective here was to master DOM manipulation in isolation: capturing user input on every keystroke, running it through validation logic, and reflecting the result back into the interface in real time without a single page reload.

Required Evidence

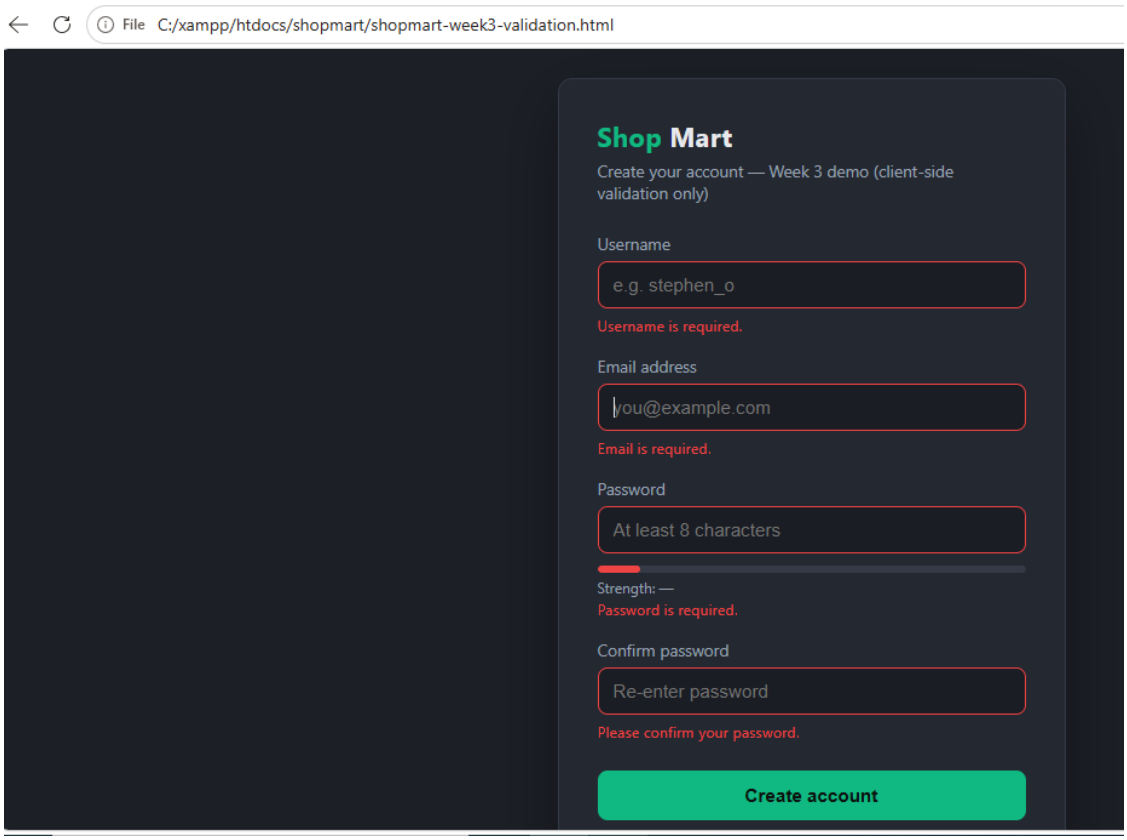


Fig 1: JavaScript Form Validation — Empty username field showing the live red error message generated by validateUsername().

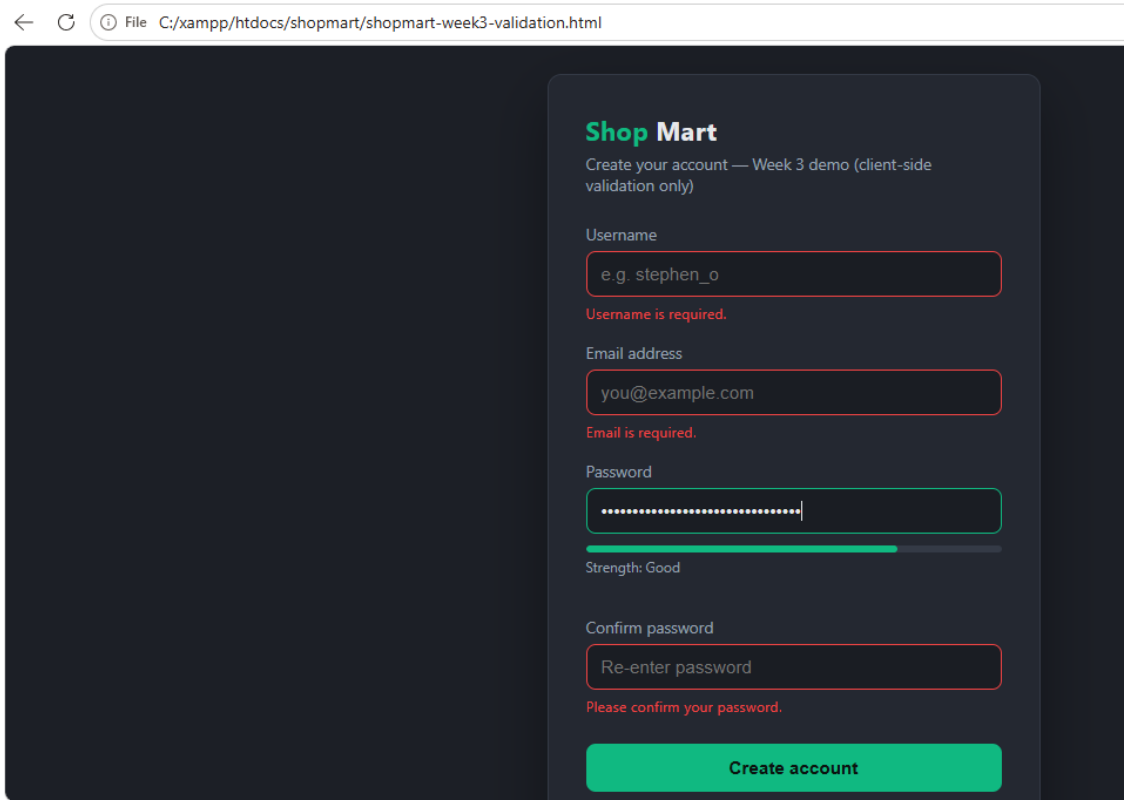


Fig 2: Password Strength Checker — Password field showing the strength meter moving from "Weak" (red) to "Strong" (green) as a longer password with symbols is typed.

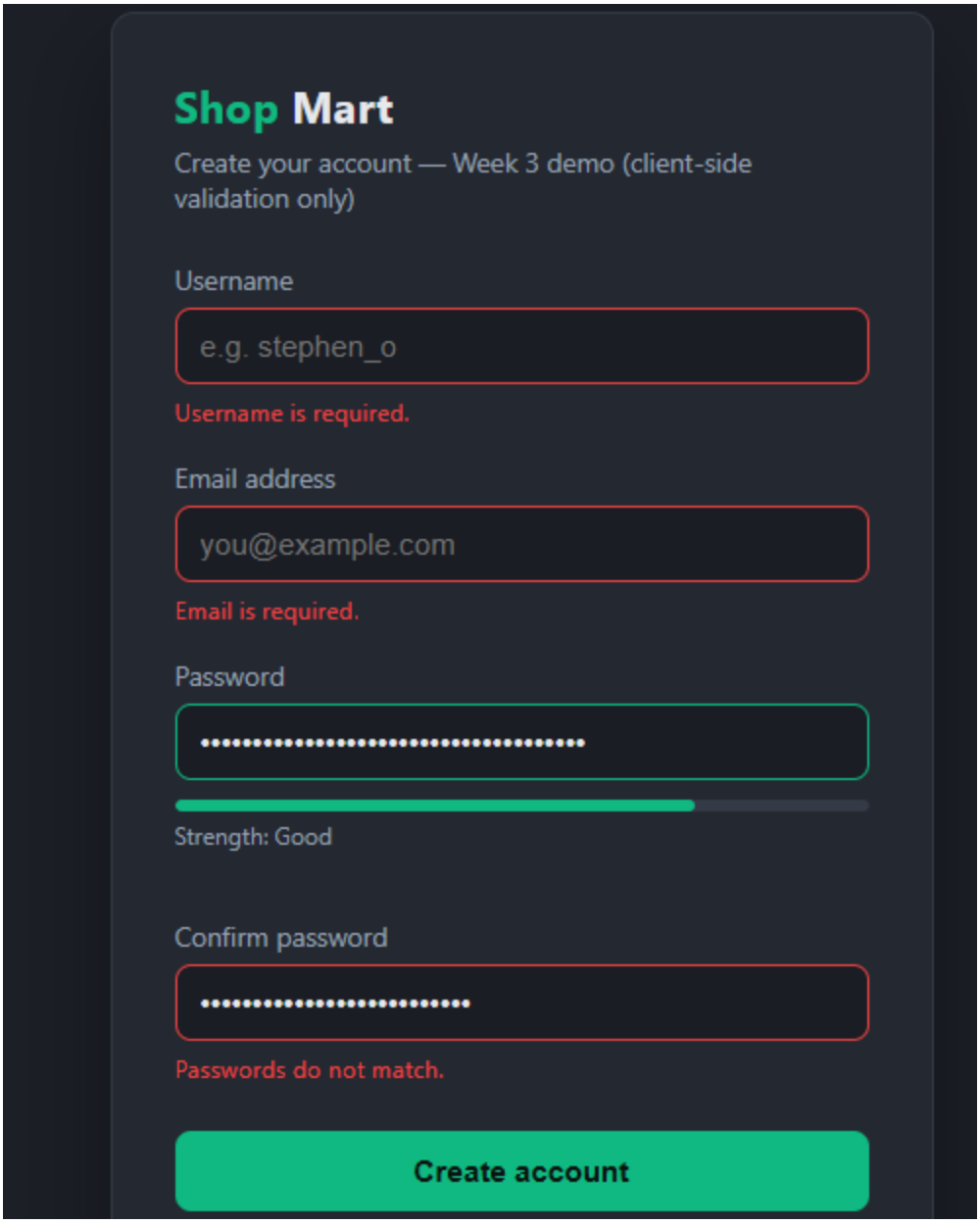


Fig 3: Confirm-Password Mismatch — Confirm Password field showing the "Passwords do not match" error when the two password fields differ.

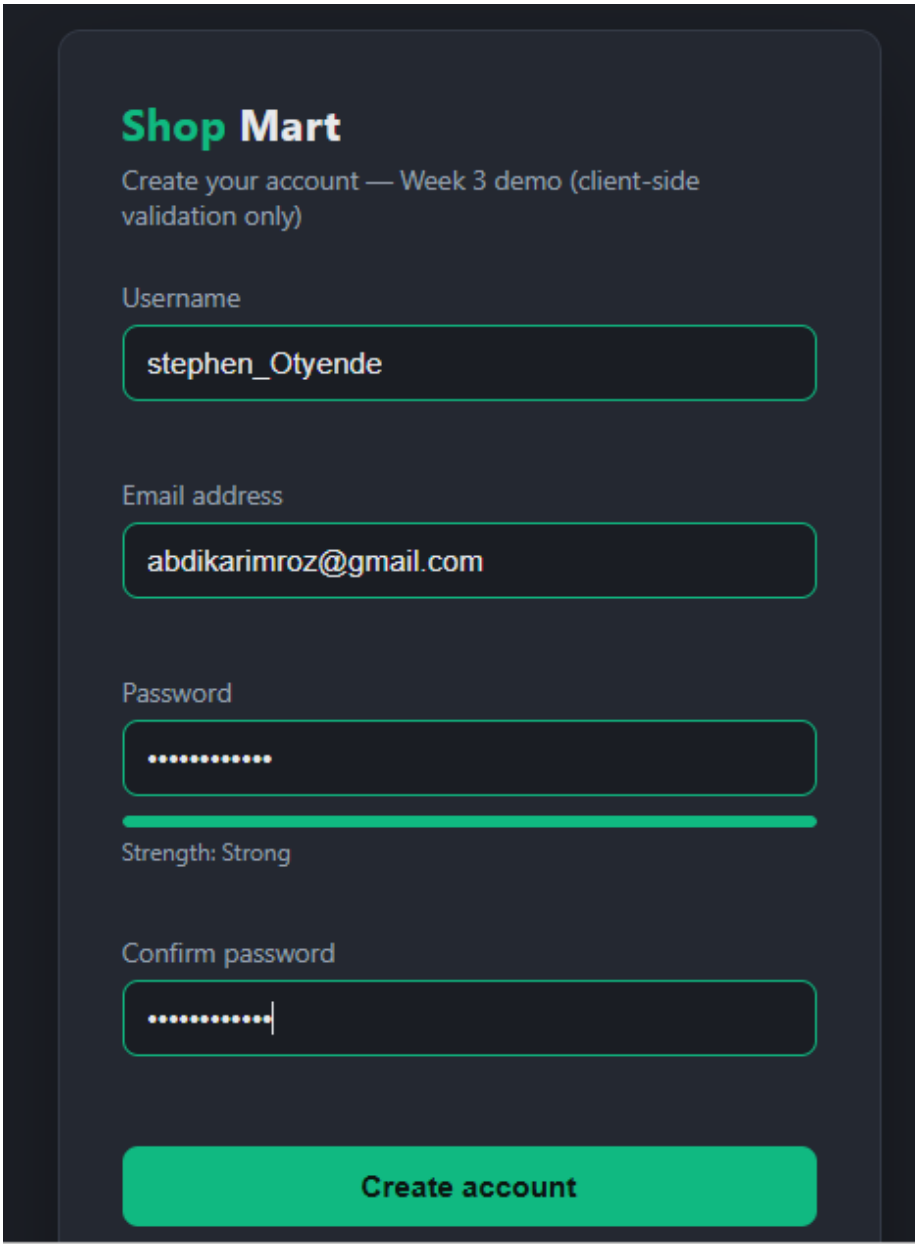


Fig 4: All Fields Valid — Completed form with all fields outlined in green and the success banner confirming the data is ready for backend submission.

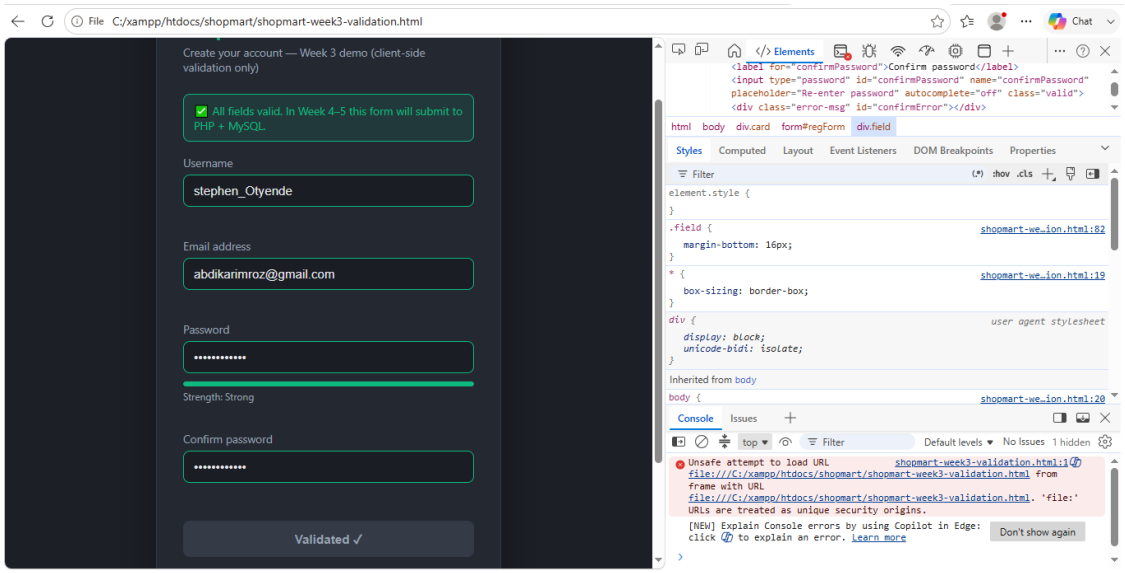


Fig 5: Browser DevTools — DOM Inspection — Chrome DevTools Elements panel showing the valid/invalid classes being added to input elements live, confirming DOM manipulation is working as coded.

Student Reflection

> *This week clarified exactly how much validation logic can — and arguably should — happen before a single byte reaches a server. The registration form checks username presence, email format, password strength, and confirm-password agreement instantly through DOM manipulation, with no page reload at any point. The password-strength meter was the most demanding piece to get right, since it has to evaluate several character-class conditions concurrently rather than apply one simple pass/fail rule. The relevance to ShopMart is more than convenience: catching invalid input at the client means the PHP backend will receive substantially fewer malformed or incomplete requests once this form is wired up to MySQL in the coming weeks, which keeps server-side validation focused on security rather than basic data hygiene.*

Tools Used

- VS Code — writing and editing HTML/CSS/JavaScript
- Google Chrome + DevTools — testing validation logic and inspecting the DOM
- Browser (offline, no server required) — running the standalone HTML file

Week 4: User Authentication

Basic Login System Connected to MySQL

Title

Basic PHP Login System with MySQL Database Connection

What Was Built

I implemented a registration and login system for ShopMart in PHP, connected to MySQL through the mysqli extension. The scope was kept deliberately narrow at this stage so the core authentication mechanics could be verified correctly before any additional features were layered on top: passwords are hashed with bcrypt via password_hash(), and every query — insert, select, or otherwise — is executed as a parameterised prepared statement rather than a concatenated string, which eliminates SQL injection as an attack vector by construction rather than by input filtering. On a successful login, PHP starts a native session and persists the authenticated user's id and username, which a protected dashboard then checks for on every request before granting access. Credential verification is performed exclusively with password_verify() against the stored hash; the plain-text password is never retained or compared directly.

Required Evidence

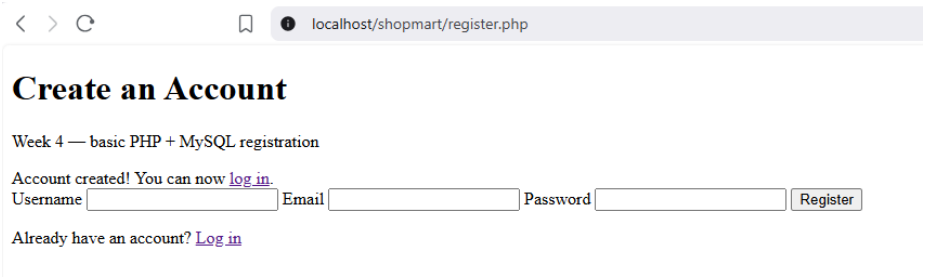


Fig 1: Registration Form & Success Message — register.php showing a filled-in registration form and the success message after submission.

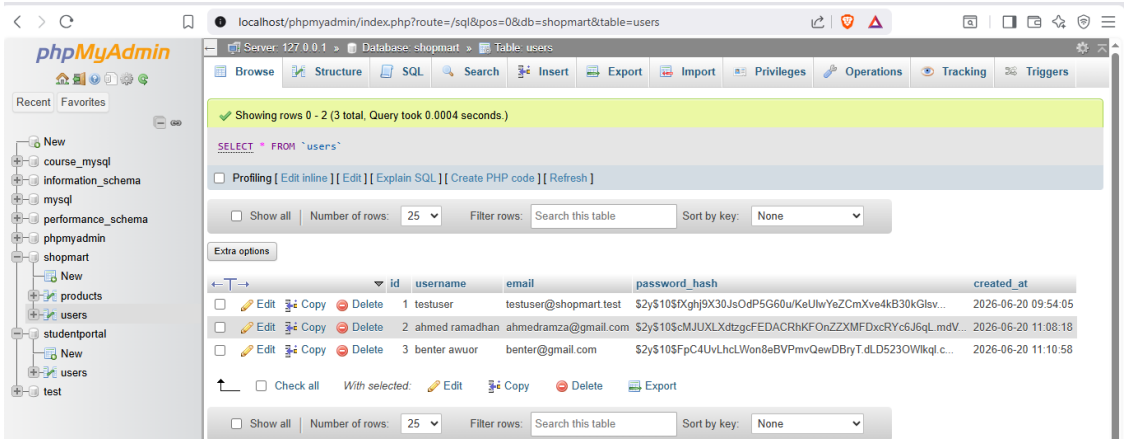


Fig 2: Hashed Password in Database — phpMyAdmin view of the users table showing the password_hash column stored as a bcrypt hash (starting with $2y$10$), never as plain text.

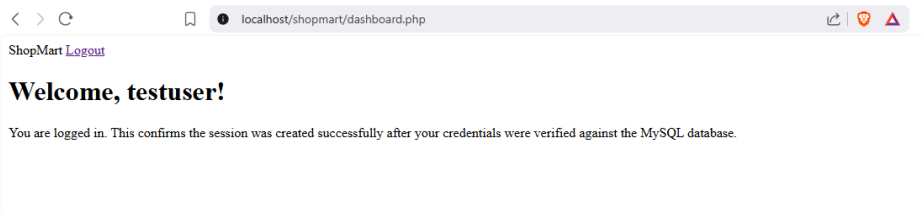


Fig 3: Successful Login & Dashboard — dashboard.php after logging in, showing a welcome message pulled from $_SESSION['username'].

Fig 4: Invalid Login Attempt — login.php showing the "Invalid username or password" message when incorrect credentials are submitted.

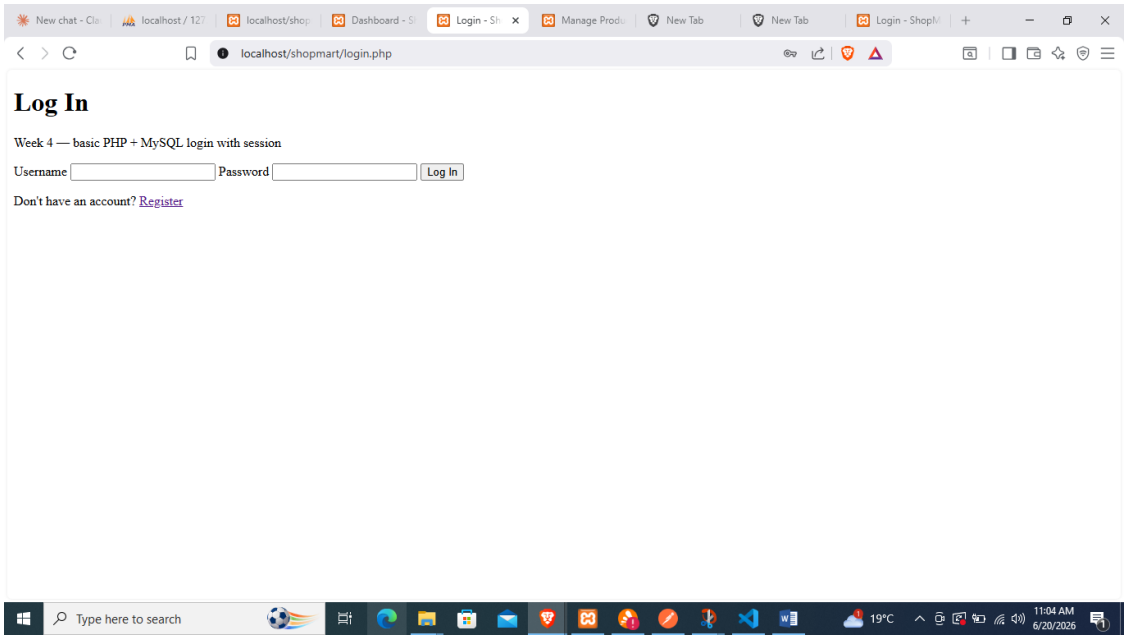


Fig 5: Logout Clearing the Session — Browser redirecting back to login.php after clicking Logout, confirming the session was destroyed.

Student Reflection

> *This was the first week the frontend genuinely talked to a persistent backend rather than simulating one. Working with mysqli's prepared statements made concrete a principle I had previously only understood in the abstract: placeholder binding keeps user-supplied data structurally separate from the SQL the database actually executes, which is why it closes off injection rather than merely filtering for it. The part that took the most iteration was coordinating password_hash() and password_verify() correctly — specifically, internalising that the hash is generated exactly once, at registration, and must never be regenerated or recomputed at login, only compared against. Getting this end-to-end flow working matters for ShopMart because it establishes a verified authentication baseline; any additional security I*

> *add later, such as rate-limiting or account lockout, sits on top of a mechanism I already know behaves correctly.*

Tools Used

- XAMPP — local Apache web server and MySQL database
- phpMyAdmin — inspecting the users table and verifying stored data
- VS Code — writing the PHP scripts

Week 5: Database Components

Full CRUD Operations for Product Management

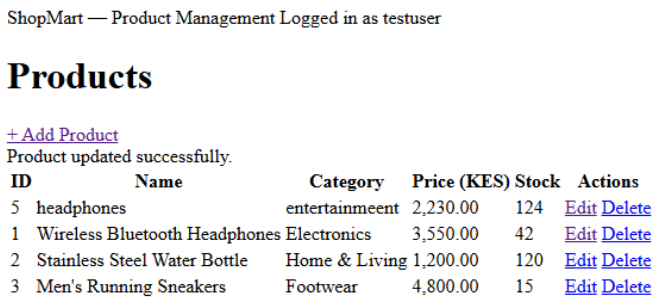
Title

Product Management Module with Full CRUD Operations (Create, Read, Update, Delete)

What Was Built

I built a complete product management module for ShopMart, giving an authenticated user full Create, Read, Update, and Delete control over product records stored in MySQL. Rather than duplicating authentication logic, the module reuses the session guard established in Week 4, so access control for the admin-facing product pages is enforced from a single, already-tested source of truth. All four operations are implemented as mysqli prepared statements with bound parameters, maintaining the same separation between user input and SQL structure established in the previous week. The product list re-queries and re-renders automatically after every Create, Update, or Delete action, with a confirmation banner surfaced so the outcome of each action is unambiguous to the user.

Required Evidence


ShopMart — Product Management Logged in as testuser

Products

+ Add Product

Product updated successfully.

ID	Name	Category	Price (KES)	Stock	Actions
5	headphones	entertainmeent	2,230.00	124	Edit Delete
1	Wireless Bluetooth Headphones	Electronics	3,550.00	42	Edit Delete
2	Stainless Steel Water Bottle	Home & Living	1,200.00	120	Edit Delete
3	Men's Running Sneakers	Footwear	4,800.00	15	Edit Delete

Fig 1: Product List (Read) — index.php showing the products table loaded from MySQL, with Edit and Delete links for each row.

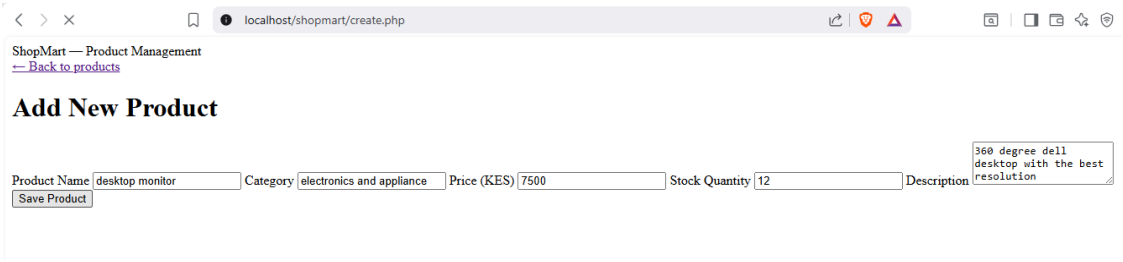


Fig 2: Adding a Product (Create) — create.php form filled in with a new product's details, and the resulting "Product created successfully" message.

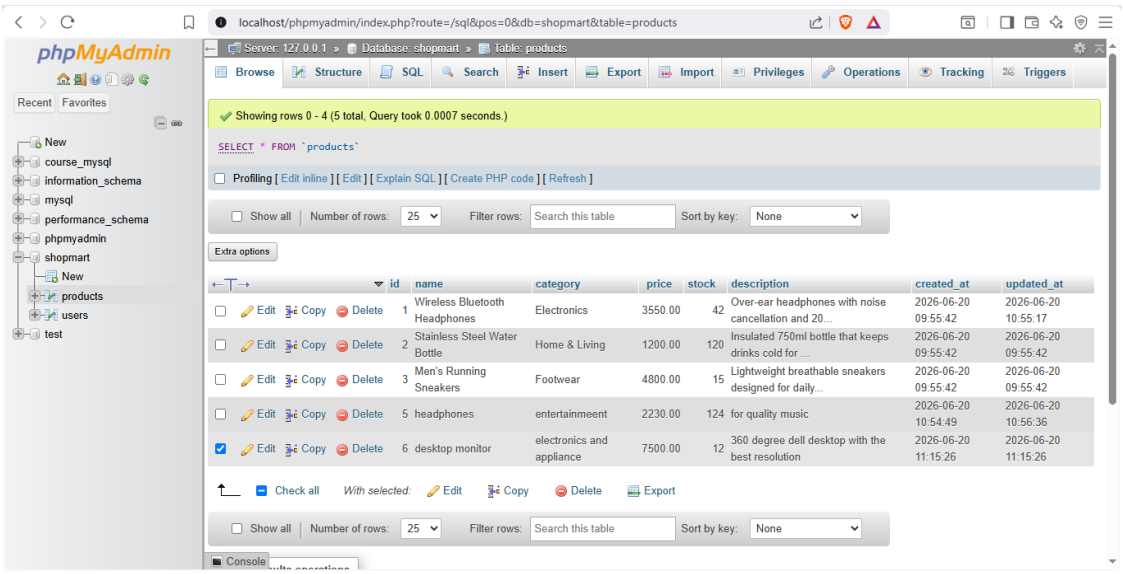


Fig 3: New Product Confirmed in phpMyAdmin — phpMyAdmin view of the products table showing the newly inserted row matching the form data.

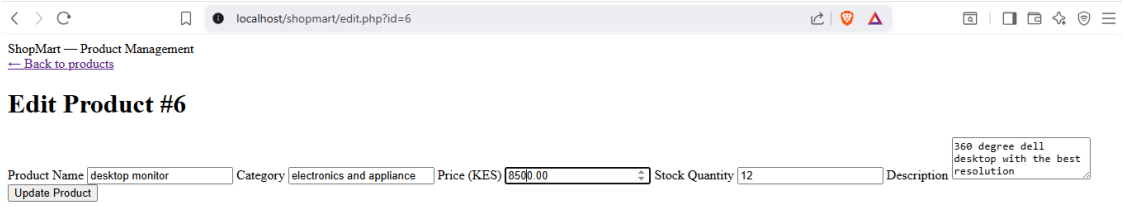


Fig 4: Editing a Product (Update) — edit.php pre-filled with an existing product's data, with the price or stock field changed, and the "Product updated successfully" confirmation.

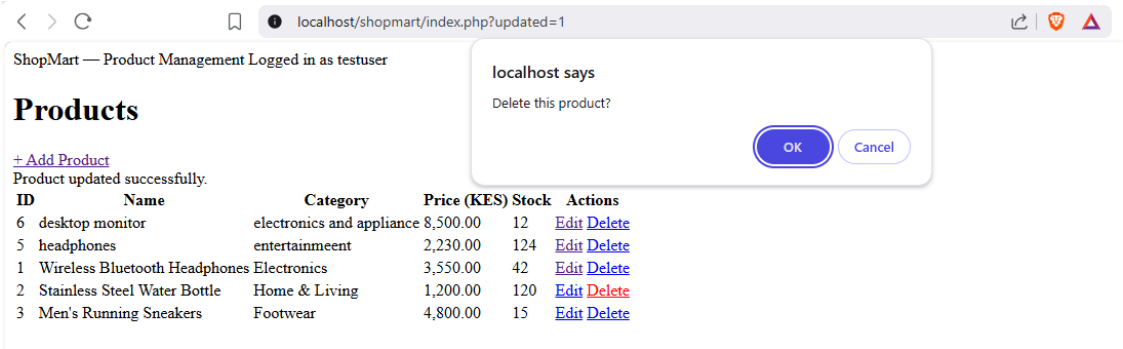


Fig 5: Deleting a Product (Delete) — The confirm() dialog before deletion, followed by the updated product list showing the row removed and the "Product deleted" message.

Student Reflection

> *This week brought together everything from the previous four into something an administrator could genuinely use — a working product catalogue backed by MySQL rather than a series of isolated demonstrations. The UPDATE statement caused the most difficulty: mysqli's bind_param() requires the type specifier string to match the bound values exactly, and a mismatch fails silently rather than throwing a usable error, which made debugging considerably harder than it should have been. Reusing the Week 4 session guard instead of rewriting it also reinforced a lesson about code organisation — factoring shared*

> *logic into a single included file means a fix or change only has to be made once. The relevance to ShopMart is structural: product management is effectively the core of the admin experience, since every other feature, from the storefront listing to future order processing, depends on this data being correct.*

Tools Used

- XAMPP — local Apache web server and MySQL database
- phpMyAdmin — verifying inserted, updated, and deleted product records
- VS Code — writing the PHP CRUD scripts

Part B — Week 6: Database Integration and CRUD Operations

In-class activities built around the studentdb example, followed by the Week 6 challenge task: a complete Employee Records Management System combining authentication, search, and full CRUD functionality.

Week 6: Database Integration and CRUD Operations

Employee Records Management System

This week centred on connecting PHP to MySQL and implementing full CRUD (Create, Read, Update, Delete) functionality end to end. I worked through the three in-class activities first, using the studentdb example set out in the lecture to consolidate the basic pattern, before building the Week 6 challenge task — a complete Employee Records Management System — as my main practical submission. Both projects use prepared statements consistently for every query and escape all output rendered back into HTML, and both were tested against a local PHP and MySQL server before the screenshots below were captured.

Class Activity 1 — Student Registration Form

Requirement: build a form that collects Full Name, Email, and Course, and persists it to MySQL. I implemented activity1_register.php with server-side validation — required-field checks combined with a genuine email-format check rather than a superficial one — and used a prepared INSERT statement to guard against SQL injection from the outset. A clear success message confirms each saved record, and empty or malformed fields surface friendly inline errors rather than allowing the script to fail silently or return a blank page.

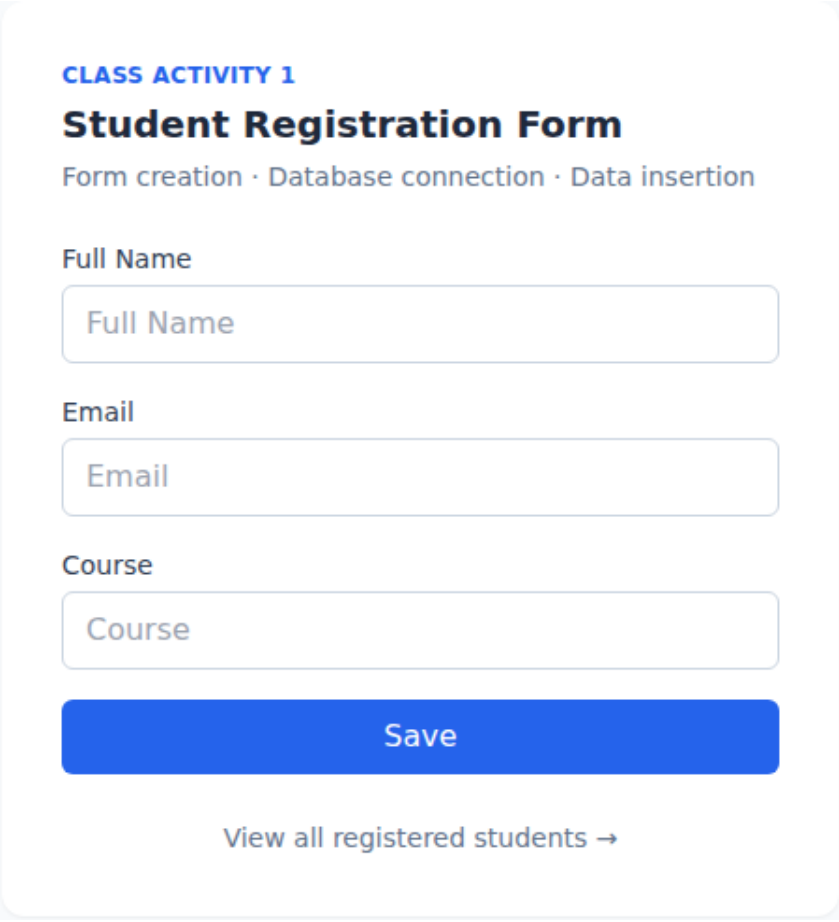


Figure 1: Student registration form (Class Activity 1)

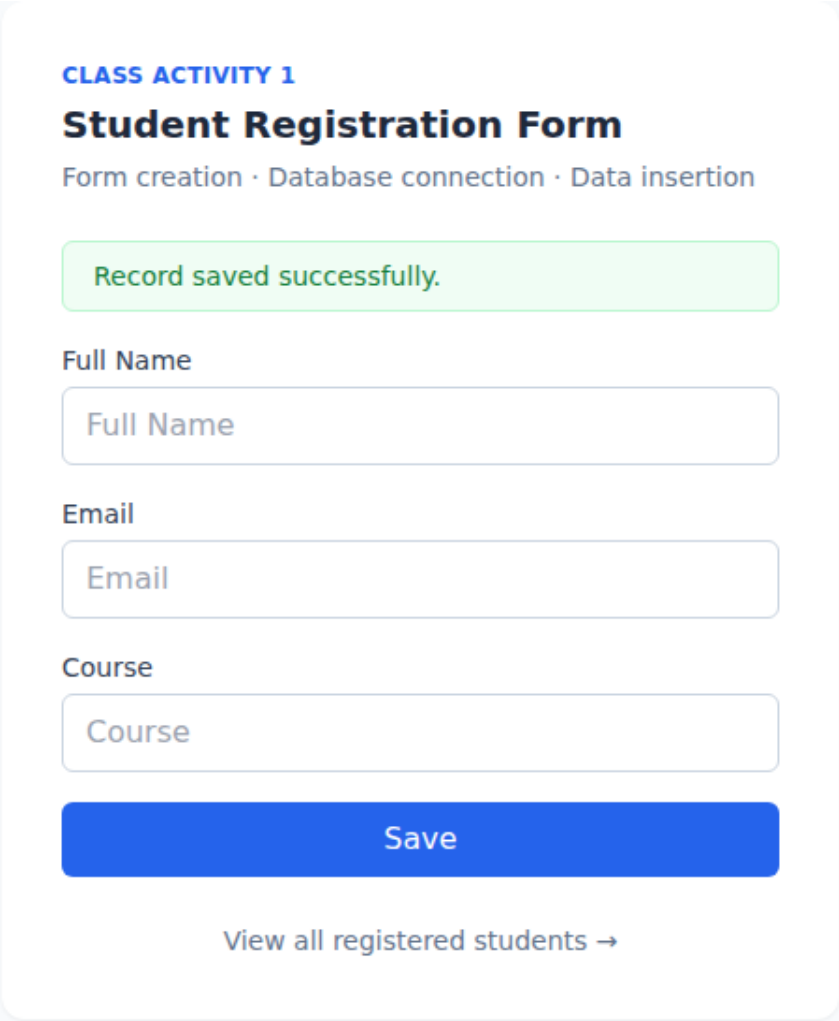


Figure 2: Successful registration — confirmation message shown, form cleared

Class Activity 2 — Display Student Records

Requirement: retrieve every record from the students table and render it as a table. activity2_display.php executes a SELECT * query ordered with the most recent entries first, then iterates the result set using fetch_assoc() to build the table rows. Each row links directly through to the management page, which let me move straight from viewing a record into editing or deleting it without an intermediate step.

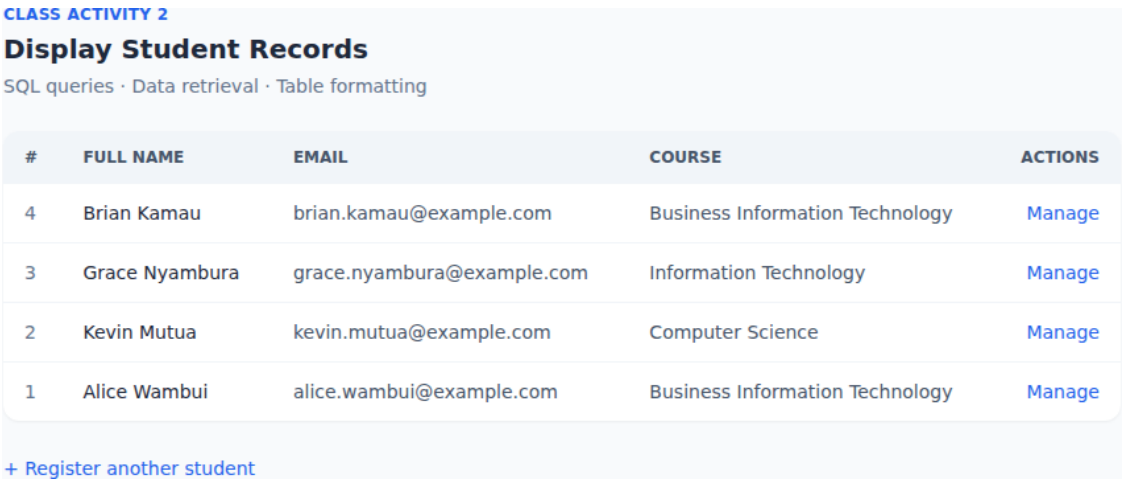


Figure 3: All registered students displayed in a table (Class Activity 2)

Class Activity 3 — Edit and Delete Records

Requirement: update a student's course and delete a selected record. activity3_edit_delete.php handles both operations as parameterised prepared statements (UPDATE ... WHERE id = ? and DELETE ... WHERE id = ?), keeping the same injection-safe pattern used throughout the rest of the unit. Selecting Edit loads the corresponding student into an inline form above the table, while Delete requires explicit confirmation first, so a record cannot be removed by an accidental click. Both actions redirect back to the listing page with a status message rather than terminating on a bare, unstyled response.

Figure 4: Edit/Delete actions available on every row (Class Activity 3)

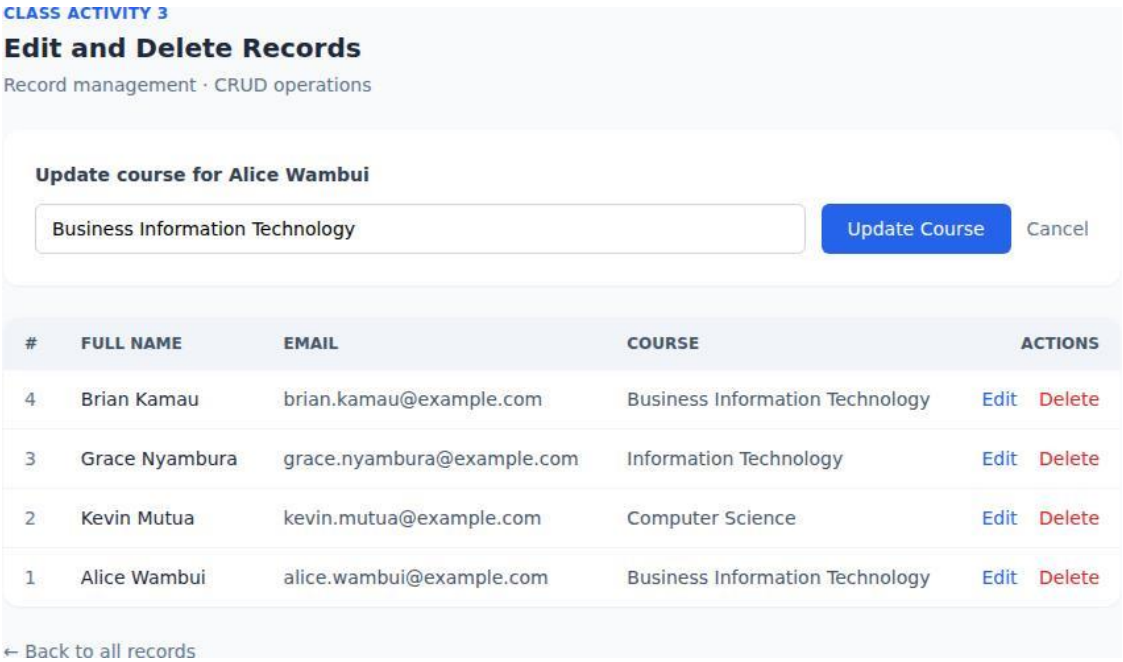


Figure 5: Inline edit form pre-filled with the selected student's course

Practical Task 3 (Challenge Task) — Employee Records Management System

This is my main Week 6 submission. Rather than completing the basic Student Management System and Library System as two separate, smaller exercises, I opted to build the Challenge Task in full, including all four bonus features named in the brief: search, authentication, responsive layout, and form validation.

Login and session protection

Every page other than login.php checks for an active session via a shared includes/auth.php guard and redirects back to the login screen if none exists. Passwords are hashed with bcrypt through password_hash() and password_verify() — at no point is a password stored, logged, or compared in plain text.

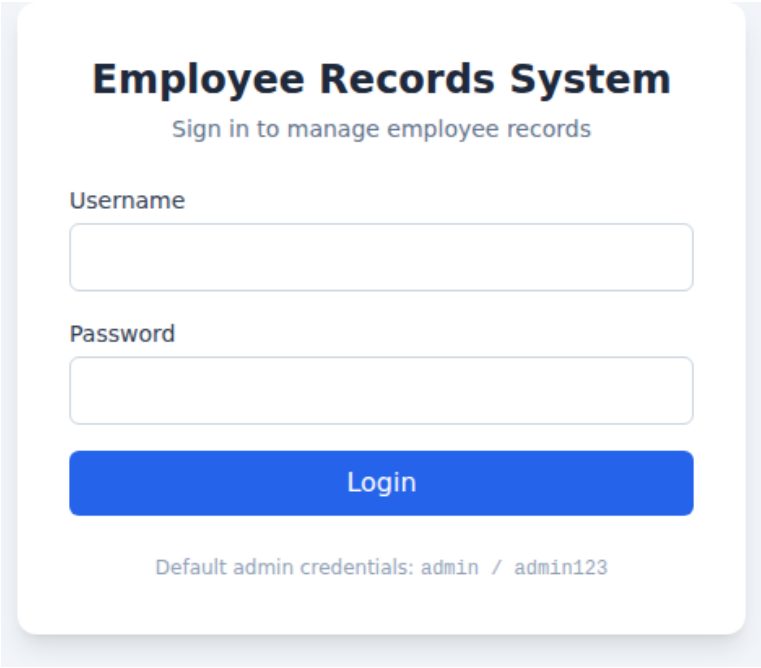


Figure 6: Login screen for the Employee Records System

Read — dashboard with search

On successful login the user lands on the dashboard, which lists every employee alongside their department, position, salary (in KES), and date hired. The search box filters this list by name, department, or position using a parameterised LIKE query, so free-text search does not reopen the injection risk the rest of the system was built to avoid.

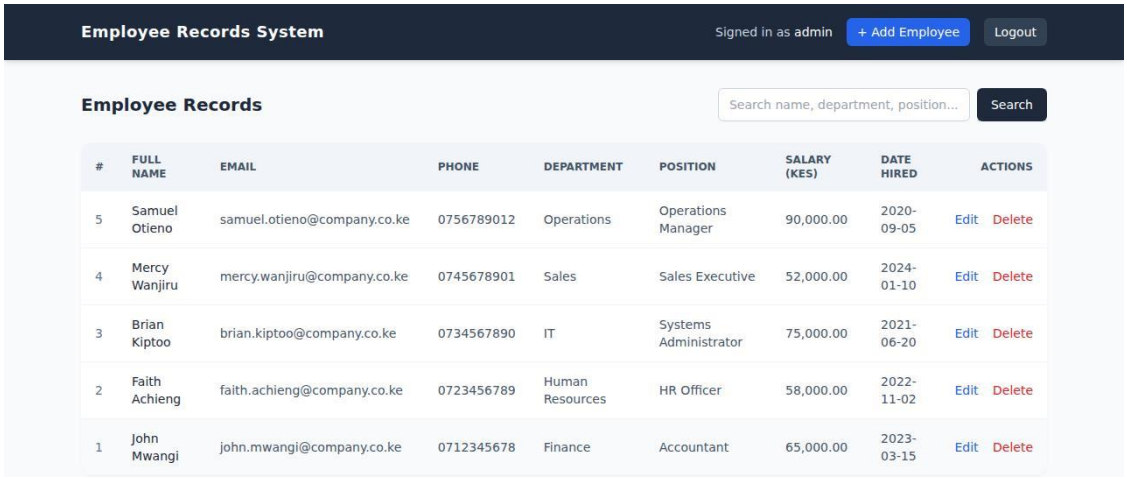


Figure 7: Employee dashboard after logging in as admin

Create — adding a new employee

add_employee.php validates every field server-side — required name, department and position, a genuine email-format check, and a numeric salary — before any INSERT is attempted, and re-populates the form with whatever was already typed if validation fails, so a single invalid field does not force the entire form to be retyped.

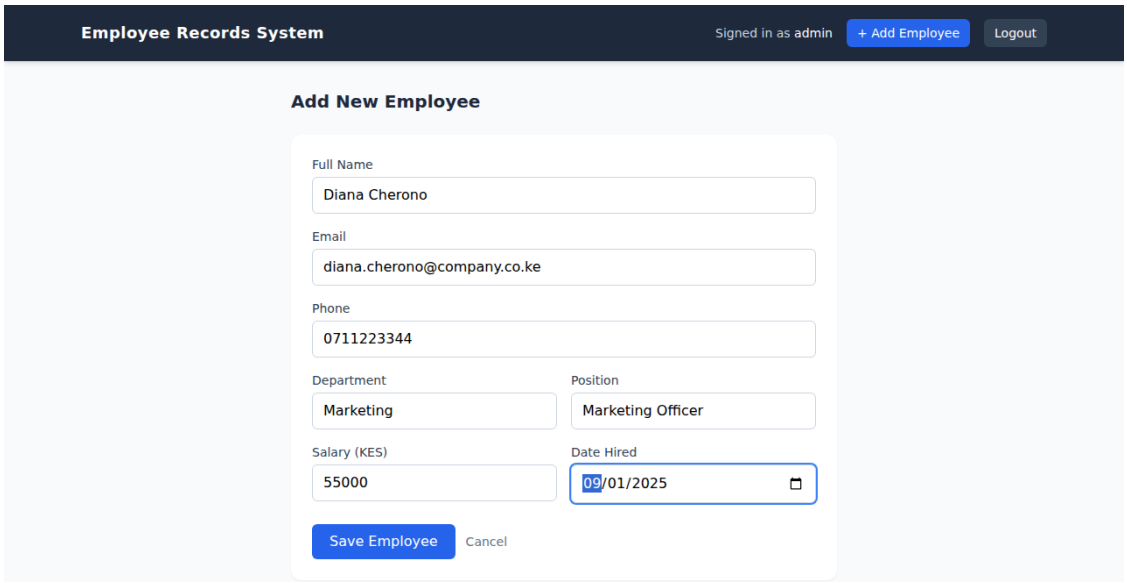


Figure 8: Add Employee form, filled in and ready to submit

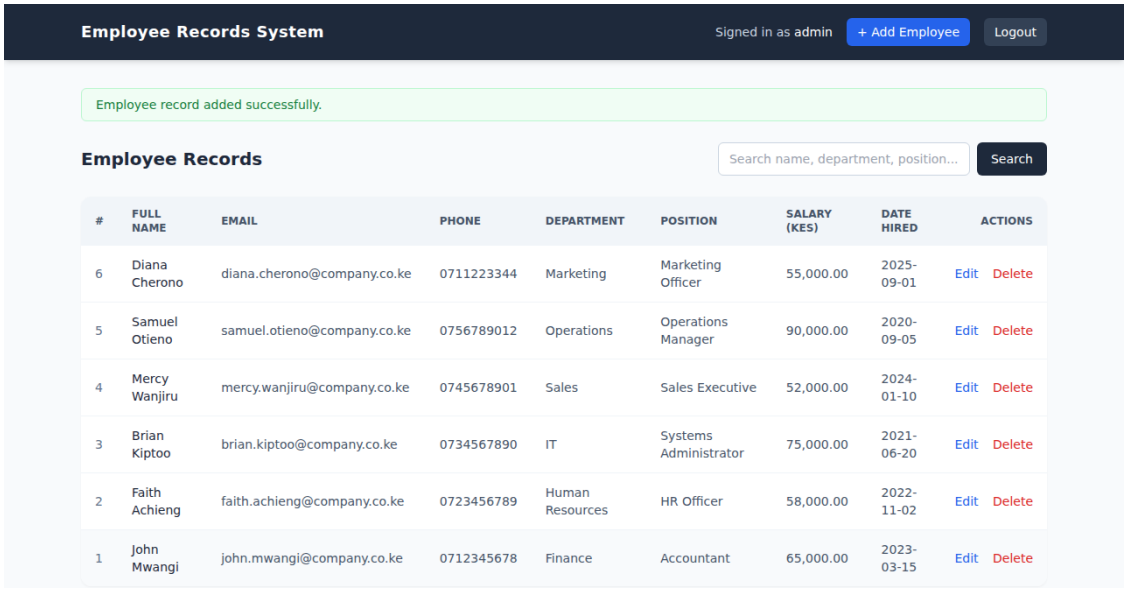


Figure 9: New employee (Diana Cherono) now showing on the dashboard

Read — search in action

Searching “IT” correctly matches both the IT department and any position containing that substring, which is the partial-match search behaviour the brief specifically asked for.

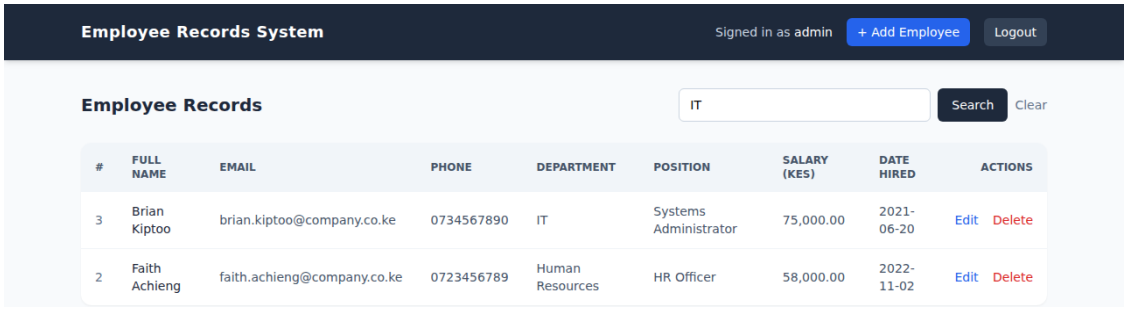


Figure 10: Search results filtered by department/position

Update — editing a record

edit_employee.php pre-loads the selected employee into the same form layout used for Add, applies identical validation rules, and commits the change with a single prepared UPDATE statement, ensuring the create and edit paths cannot drift out of sync with each other.

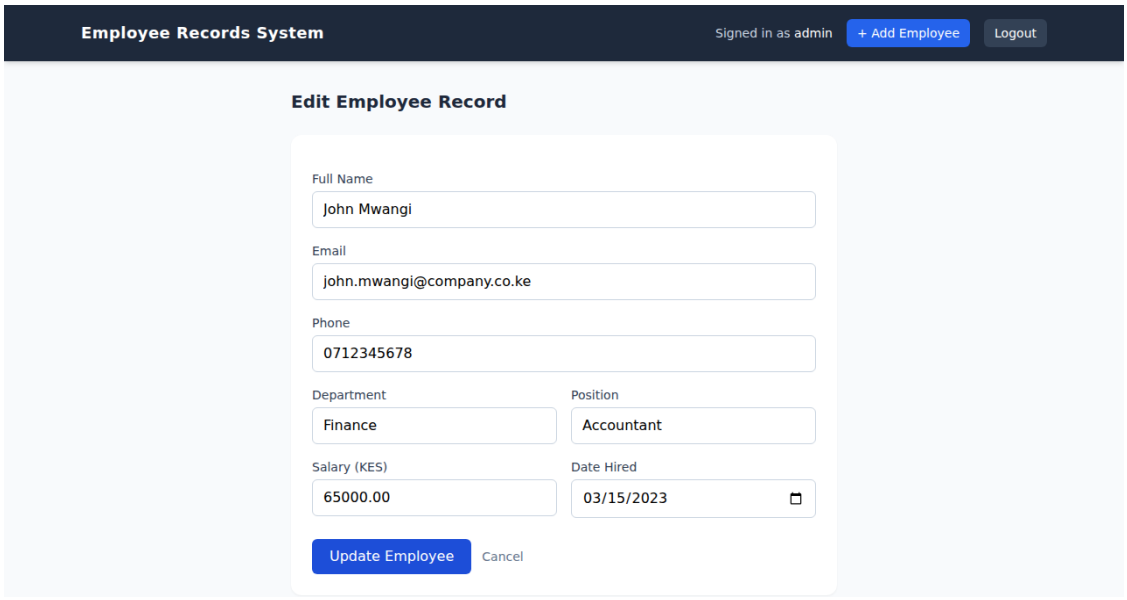


Figure 11: Edit Employee form, pre-filled with the existing record

Delete

Delete is triggered directly from the dashboard, guarded by a JavaScript confirm() prompt so a record cannot be removed by an accidental click, and the row is then removed with a prepared DELETE statement.

Weekly Reflection Questions

Why are databases important in web applications?

Databases give a web application persistent state. Without one, any data submitted — student records, employee records, or anything else — would exist only for the lifetime of a single HTTP request and disappear the moment the page reloads; a database lets that data be retrieved, updated, or deleted in later, entirely separate requests.

Static vs dynamic websites:

A static site serves fixed HTML that is identical for every visitor and every request. A dynamic site, such as both projects built this week, generates its markup from live database queries on every page load, so the content reflects the current state of the data rather than a snapshot taken when the file was last written.

CRUD operations, with examples from my own code:

Create is handled by activity1_register.php and add_employee.php; Read by activity2_display.php and index.php; Update by activity3_edit_delete.php and edit_employee.php; and Delete by activity3_edit_delete.php and delete_employee.php — each operation implemented as a discrete, prepared-statement query rather than a single catch-all script.

How does PHP communicate with MySQL?

Through the mysqli extension. I open a single connection object per request, then use prepared statements with bind_param() to bind user-supplied values safely, and retrieve results with fetch_assoc(), which returns each row as an associative array keyed by column name.

Why validate user input?

So that invalid, incomplete, or malicious data never reaches the database in the first place — for example, catching a missing email address or a non-numeric salary before it gets anywhere near an INSERT statement, rather than relying on the database to reject it after the fact.

Security risks from poor database design:

The three risks I deliberately designed against this week were SQL injection, addressed by using prepared statements for every query without exception; plain-text password storage, addressed by hashing every password with bcrypt before it is ever written to the database; and unrestricted page access, addressed by enforcing a session-based login guard on every protected page rather than trusting the user not to navigate there directly.

Part C — Week 7: Student Portal Login System

A standalone practical task covering user authentication and session management, implemented independently of the ShopMart project so the underlying security mechanics could be isolated and verified on their own terms.

Week 7: User Authentication and Session Management

Practical Task 1 — Student Portal Login System

Project Title	Student Portal Login System (Standalone Practical Task)
Technologies (Week 7)	PHP (mysqli), MySQL, HTML5, CSS3, PHP Sessions
Week Topic (per Weekly Brief)	User Authentication and Session Management — Practical Task 1: Student Portal Login System

Title

Student Portal Login System — Registration, Authentication, Dashboard, and Logout

What Was Built

I built a standalone Student Portal Login System directly from this week's practical brief, deliberately kept independent of the ShopMart project so the authentication workflow could be implemented and tested in isolation. The system covers the complete cycle taught this week: a registration form that hashes passwords with password_hash() before they are written to MySQL, a login form that verifies credentials with password_verify() and starts a PHP session only on success, a protected dashboard that checks for that active session on every request before granting access, and a logout script that explicitly destroys the session. The users table stores fullname, email, and a hashed password column, matching the schema given in the weekly material, and mysqli is used throughout for the database connection, consistent with the brief's connection.php example.

Key Features Implemented

- index.html — landing page linking to Login and Register.
- register.php — registration form with server-side input validation, a duplicate-email check against existing records, and password_hash() applied before the INSERT.
- login.php — verifies submitted credentials with password_verify() and creates $_SESSION['user'] only on a successful match.
- dashboard.php — protected page; redirects to login.php if no active session is found.
- logout.php — destroys the session and redirects to login.
- users table (MySQL) — id, fullname, email, password (hashed), matching the brief's schema.

Required Evidence

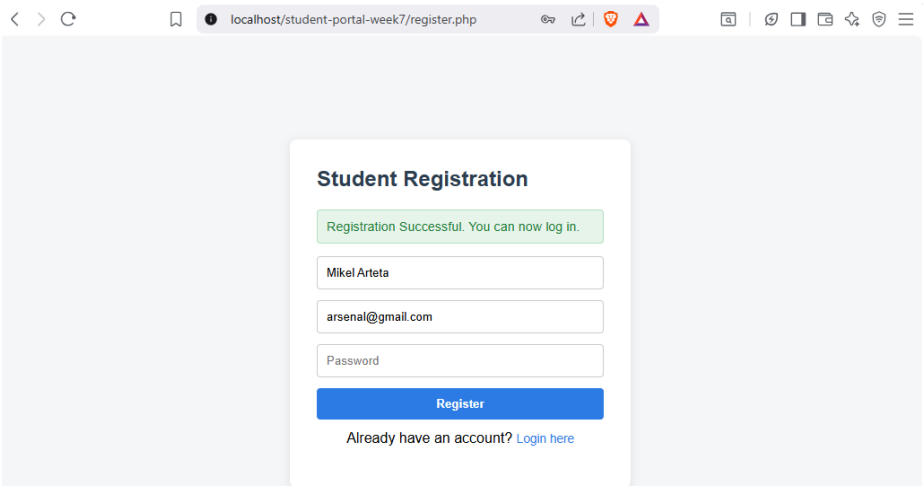


Fig 1: Registration Form & Success Message — register.php showing a filled-in registration form and the "Registration Successful" message after submission.

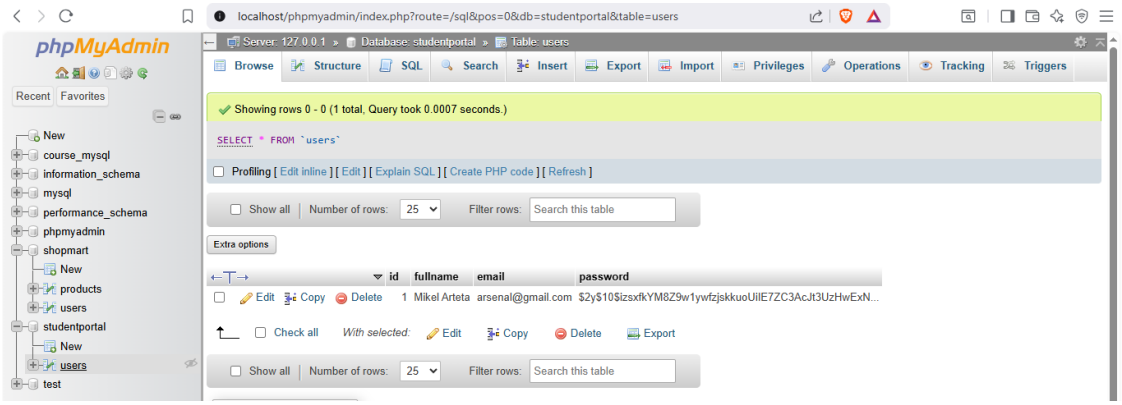


Fig 2: Hashed Password in Database — phpMyAdmin view of the users table showing the password column stored as a bcrypt hash (e.g. starting with $2y$10$), never as plain text.

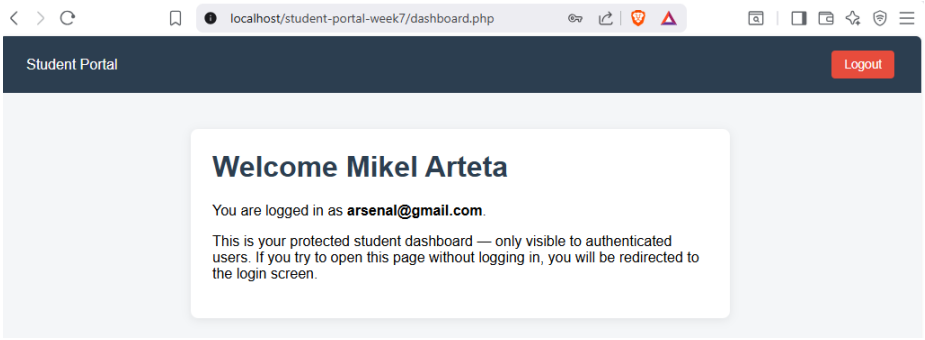


Fig 3: Successful Login & Dashboard — dashboard.php after logging in, showing "Welcome [Full Name]" pulled from $_SESSION['user'].

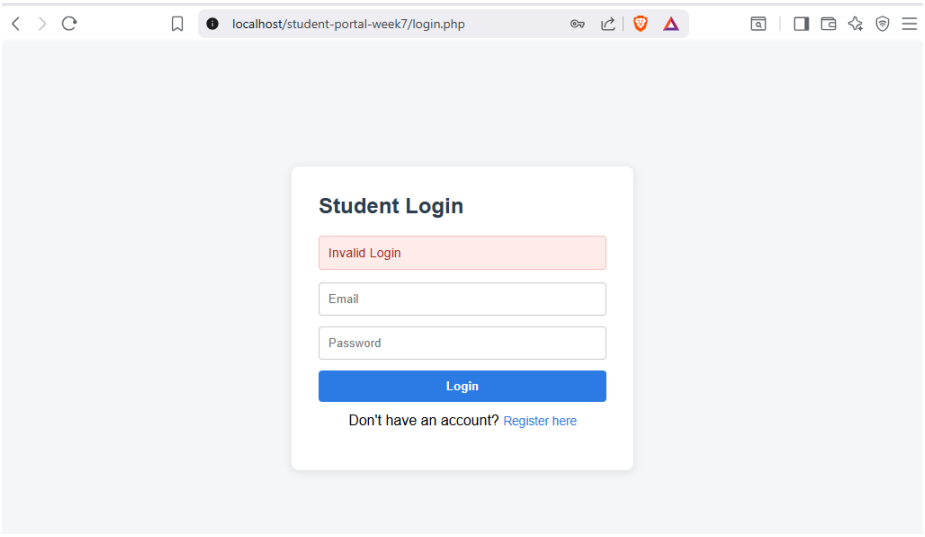


Fig 4: Invalid Login Attempt — login.php showing the "Invalid Login" message when incorrect credentials are submitted.

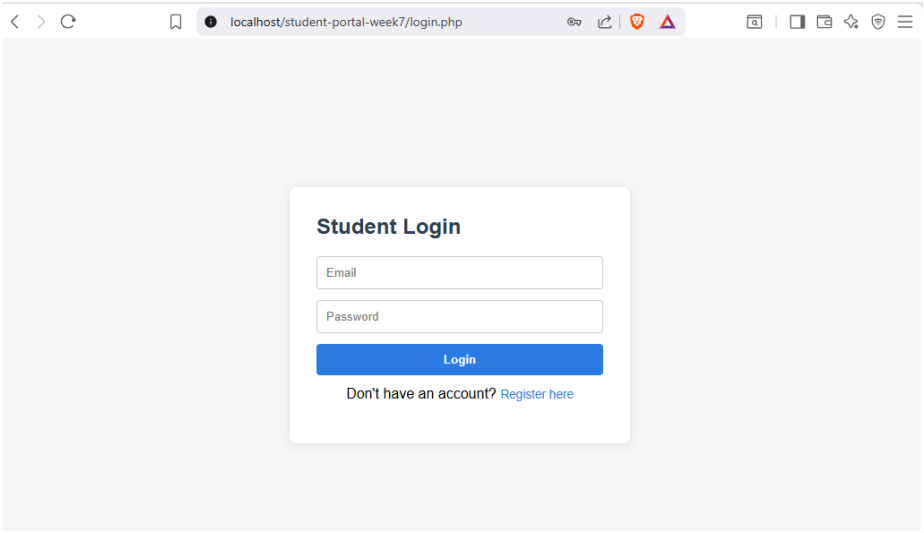


Fig 5: Protected Page Redirect & Logout — Browser address bar showing dashboard.php redirecting to login.php when accessed while logged out, and the result of clicking Logout.

Weekly Reflection Questions

What is authentication?

Authentication is the process of verifying that a user is who they claim to be — in this system, by checking a submitted email and password against the corresponding hashed record stored in the database.

How does authorization differ from authentication?

Authentication establishes identity; authorization is a separate, subsequent concern that determines what an already-identified user is permitted to do or access. A system can authenticate a user correctly and still need authorization rules to decide, for instance, whether that user may view another student's record.

Why should passwords be hashed?

Hashing converts a password into a one-way, effectively irreversible string before it is stored, so that even if the database were compromised, the original passwords would not be recoverable from what was

leaked — unlike plain-text or even simply encrypted storage, where compromise of the key would expose every password at once.

What is the purpose of sessions?

Sessions let the server retain knowledge of a logged-in user across multiple, otherwise independent page requests. HTTP itself is stateless — each request is handled with no memory of any previous one — so without a session mechanism, a server would have no way to know a user had already authenticated on the very next page they visited.

Why are protected pages important?

Protected pages ensure that sensitive content and actions are only reachable by users who have already authenticated, rather than relying on the absence of a visible link to keep a page hidden. Without an explicit session check, a page like the dashboard would be accessible to anyone who simply guessed or was given its URL.

What are the dangers of SQL injection?

SQL injection allows an attacker to manipulate the structure of a query by submitting input that the application treats as code rather than data — for example, entering ' OR '1'='1 into a login field to construct a condition that is always true, bypassing authentication entirely, or extracting and altering data the attacker should never have had access to. Prepared statements prevent this at a structural level, because the query's logic is compiled before any user-supplied value is bound to it, so that value can never be interpreted as part of the SQL itself.

How does logout improve security?

Logout destroys the session immediately rather than simply navigating the user away from it, which means the session identifier can no longer be used to access protected pages even if it were somehow intercepted. This shortens the window during which a stolen or leaked session ID would remain valid, which is the basis of session-hijacking attacks.

Student Reflection

> *This week turned authentication from a concept I could describe into a mechanism I could actually demonstrate working end to end. Building registration, login, and logout as a standalone exercise, free of any other project's surrounding complexity, made it easier to concentrate purely on the security mechanics: password hashing, password_verify(), and session validation. The detail that took the most deliberate thought was recognising that the dashboard has to check for an active session on every single load, not only immediately after login — otherwise a user who had logged out, or who never logged in at all, could still reach the page simply by typing its URL directly. This matters beyond the scope of this one task because authentication is the foundation every other feature in a web application ultimately depends on; without a reliable way to establish who is making a request, no subsequent authorization, audit, or personalisation logic can be trusted.*

Tools Used

- XAMPP — local Apache web server and MySQL database
- phpMyAdmin — verifying the users table and confirming passwords are hashed
- VS Code — writing the PHP authentication scripts

Part D — Week 8: Responsive Web Design and Mobile-First Development

Two in-class responsive design tasks, plus the extra practical assignment: a complete five-page Crumb & Co. cake training and supplies website, all designed mobile-first and validated across multiple breakpoints.

Week 8: Responsive Web Design and Mobile-First Development

Date: 22 June 2026

Objectives This Week

- Define Responsive Web Design (RWD) and explain why mobile-friendly sites matter.
- Differentiate between responsive and adaptive web design.
- Apply CSS media queries to adjust layout at different breakpoints.
- Design web pages using a Mobile-First approach.
- Build responsive layouts using Flexbox and CSS Grid.
- Test the same page across mobile, tablet, and desktop screen sizes.

This was a demanding practical week — two class tasks plus the extra assignment — so the majority of my time went into actually building and testing layouts across breakpoints, rather than reading about responsive theory in the abstract.

Task 1: Responsive Personal Profile Page

I built a personal profile page using Flexbox as the primary layout tool. On mobile, the hero is deliberately a single column — avatar on top, followed by name, role, and contact details stacked beneath it — to respect the limited horizontal space. At the 600px breakpoint, the hero switches to flex-direction: row, so the avatar sits beside the text rather than above it, and at 1024px the layout is given more breathing room throughout: a larger avatar, wider gaps between elements, and a larger type scale.

For the profile picture, I deliberately used a genuine <img> element — a small base64-encoded SVG avatar — rather than embedding an inline <svg> directly in the markup, since the brief specifically requires the profile image to be responsive. An inline SVG would visually resemble an image but would not be subject to the same max-width: 100%; height: auto; responsive-image rule, so using a real <img> element was necessary to satisfy the requirement rather than merely appear to.

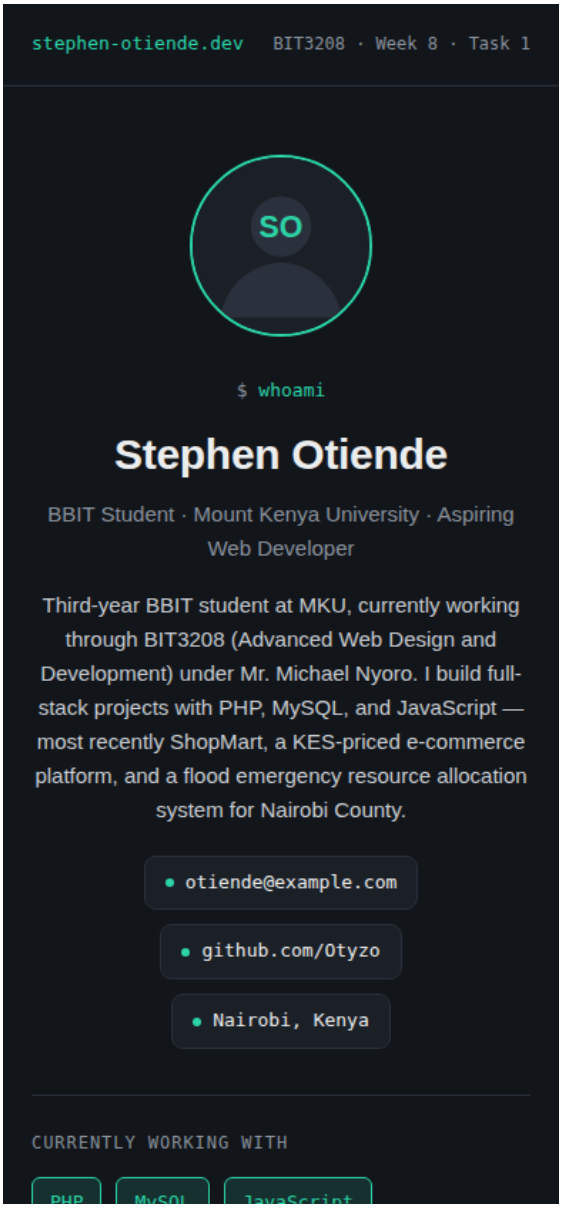


Figure 8.1: Profile page — mobile view (390px), single-column layout

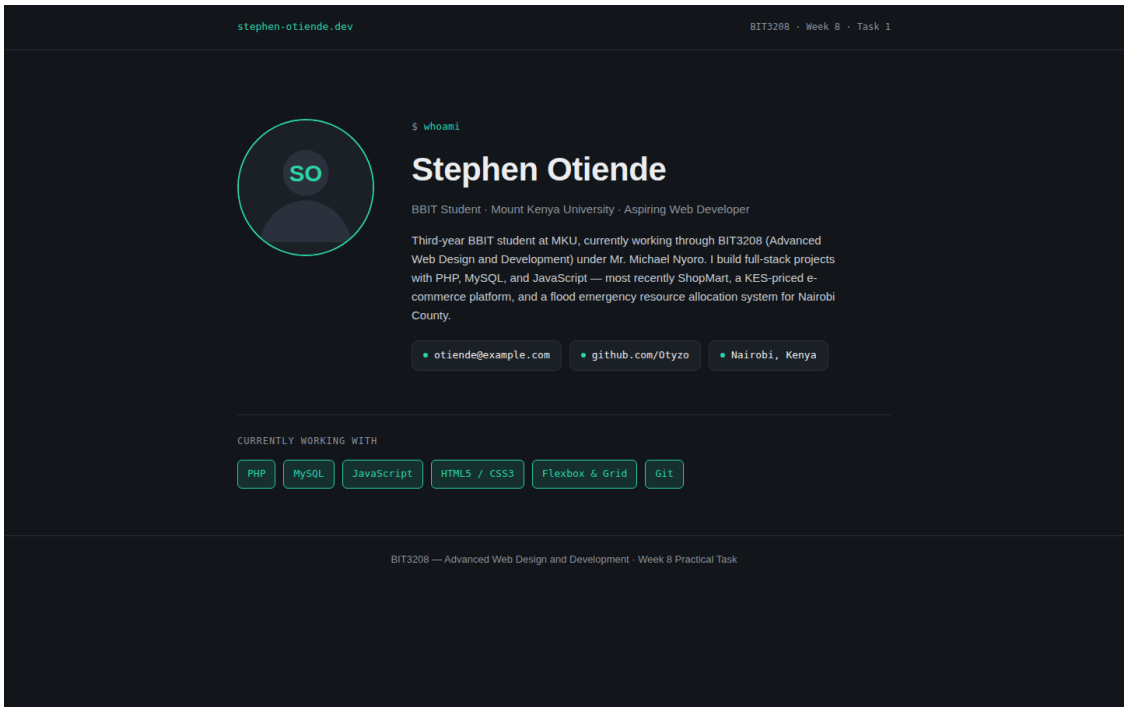


Figure 8.2: Profile page — desktop view (1440px), hero switches to a row

Reflection

> *The hero's transition from column to row at the breakpoint was more straightforward than I had anticipated — a single flex-direction change inside the media query was sufficient. The contact chips were considerably more demanding: on mobile I wanted them centred and stacked vertically, while on desktop they needed to sit inline beside the bio, which meant introducing flex-wrap and reconsidering how the parent container's align-items property behaved at each breakpoint rather than treating it as a fixed setting.*

Task 2: Responsive Product Showcase

This task uses CSS Grid rather than Flexbox, since the brief specifically calls for a grid-based showcase. I built it mobile-first: grid-template-columns: 1fr as the base case, widening to repeat(2, 1fr) at 600px and repeat(4, 1fr) at 1024px, which satisfies the two required breakpoints. I also captured a tablet-width screenshot in between the two specified breakpoints, to confirm the layout held up at an intermediate width and not only at the two edges I was explicitly tested against.

I themed the showcase as a small ShopMart spin-off, reusing the same KES pricing convention and dark charcoal / emerald colour scheme established in the main build, across four products: wireless earbuds, a smart watch, a canvas tote bag, and a desk lamp. As in Task 1, each product icon is a genuine <img src="..."> element rather than an inline <svg>, so that the responsive-image rule is meaningfully constraining a real image element rather than having no effect.

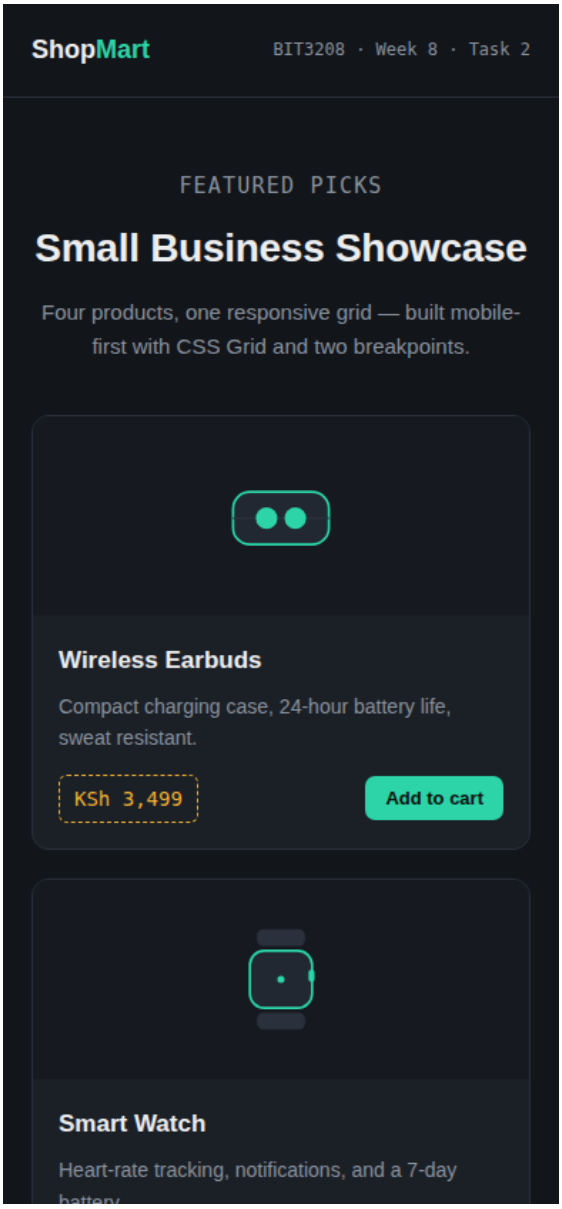


Figure 8.3: Product showcase — mobile view (390px), one column

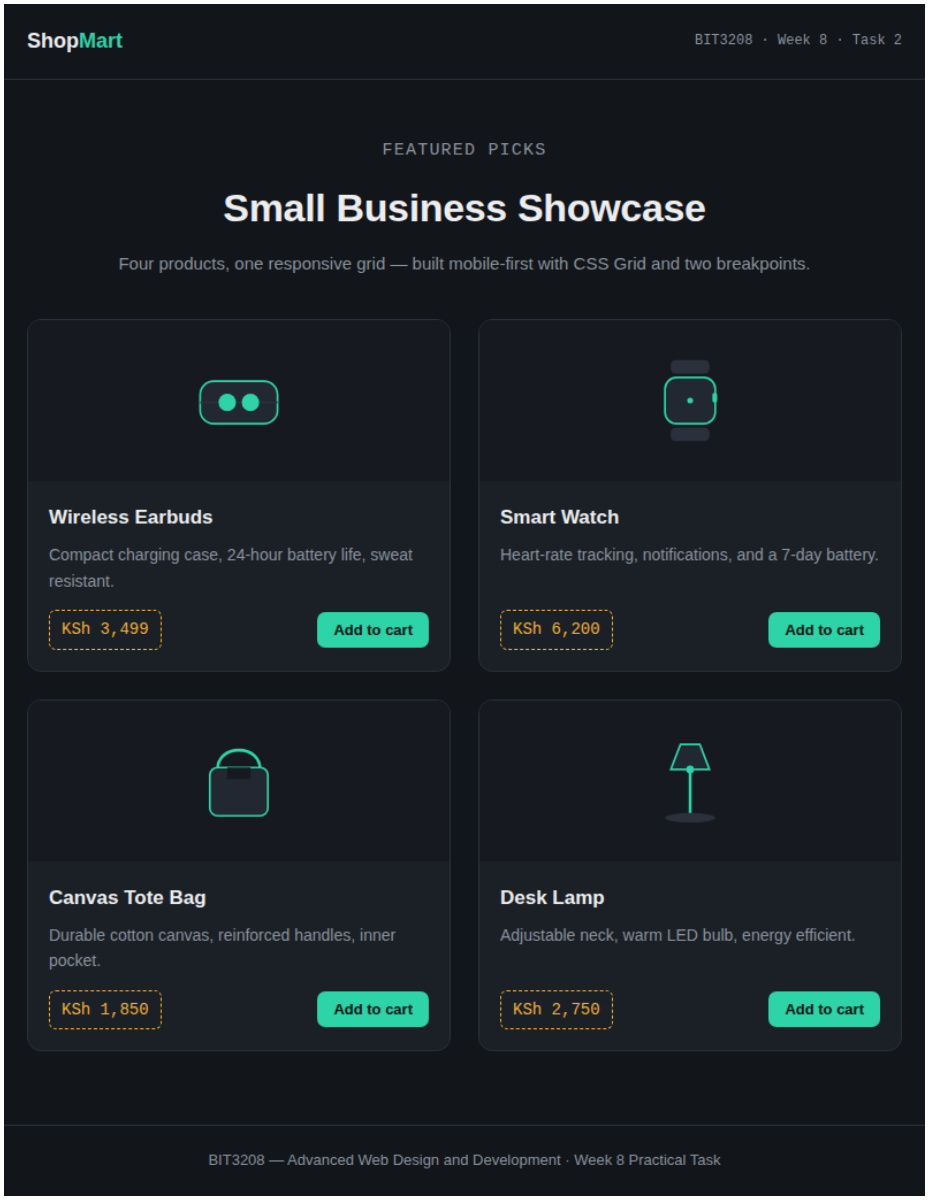


Figure 8.4: Product showcase — tablet view (800px), two columns

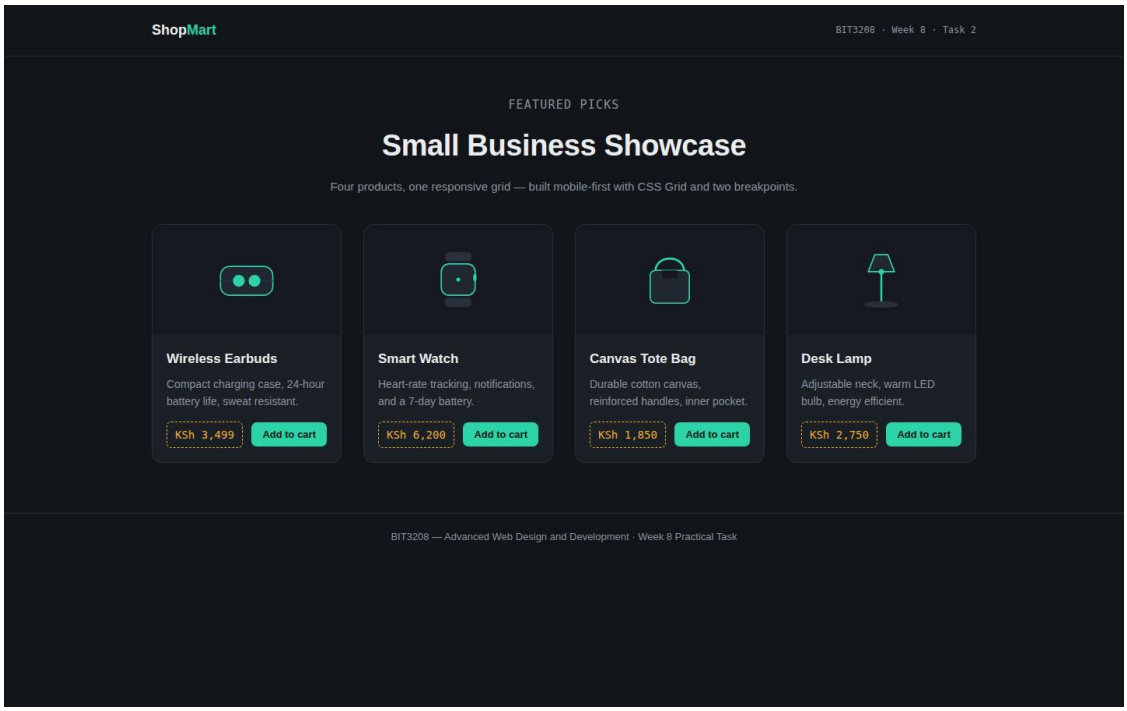


Figure 8.5: Product showcase — desktop view (1440px), four columns

Reflection

> *CSS Grid required far less manual adjustment than I had expected — changing the column count at each breakpoint is a single declaration, and the cards reflow automatically without any additional repositioning logic. The detail that did need attention was card height: applying flex-grow: 1 to the description element inside each card ensures every card in a row remains the same height, even when one product's description is noticeably longer than its neighbours'.*

Extra Practical Assignment: Crumb & Co. Cake Training & Supplies Website

I built a complete five-page site for a fictional cake-decorating training and supplies business — Home, About Us, Products, Training, and Contact — all sharing a single design system, so the result reads as one coherent product rather than five independently styled pages.

I deliberately moved away from the dark charcoal and emerald palette used for ShopMart and the two tasks above, since a cake-decorating business warranted its own distinct visual identity. I settled on a cocoa-brown background, a gold accent colour, and a soft icing-pink for price tags, and introduced a scalloped “piped icing” divider — achieved with a repeating CSS radial-gradient rather than a background image — as a single recurring signature detail between sections, in place of a plain horizontal rule.

Every page was built mobile-first in the same order: a single column with the navigation collapsed behind a hamburger button as the base case, widening at 700px for tablet (navigation still collapsed, but content expands to two columns) and at 900px for desktop, where the navigation switches to a full horizontal menu. The hamburger toggles an .open class with a small amount of vanilla JavaScript, and the desktop media query forces the menu visible regardless of that class's state, which prevents the menu from becoming stuck mid-resize if the viewport crosses the breakpoint while the menu happens to be open.

Flexbox is used for the hero section, the footer's column layout, and the split between the info panel and form on the Contact page — all genuinely one-dimensional arrangements. CSS Grid is reserved for the two-

dimensional layouts: the product cards (one, then two, then three columns) and the course cards (one, then two columns) on the Products and Training pages respectively.

The brief specifies that the site must be "validated and tested on at least three different screen sizes," so I treated validation and testing as two distinct, deliberate steps rather than relying on how the layout looked in a single browser window:

- Tested: captured the Home page at 390px (mobile), 800px (tablet), and 1440px (desktop) — Figures 8.6–8.8 — and additionally screenshotted the Products page at tablet and desktop widths specifically to verify the grid reflow (Figures 8.10–8.11).
- Validated: ran all five site pages, plus both task pages, through an HTML parser/validator rather than relying on visual inspection alone. This caught two genuine issues — an unescaped & character in the Google Fonts URL that needed to be written as &, and missing alt attributes and a lang attribute on elements where they had been omitted — both of which I corrected. All seven pages now parse without errors.

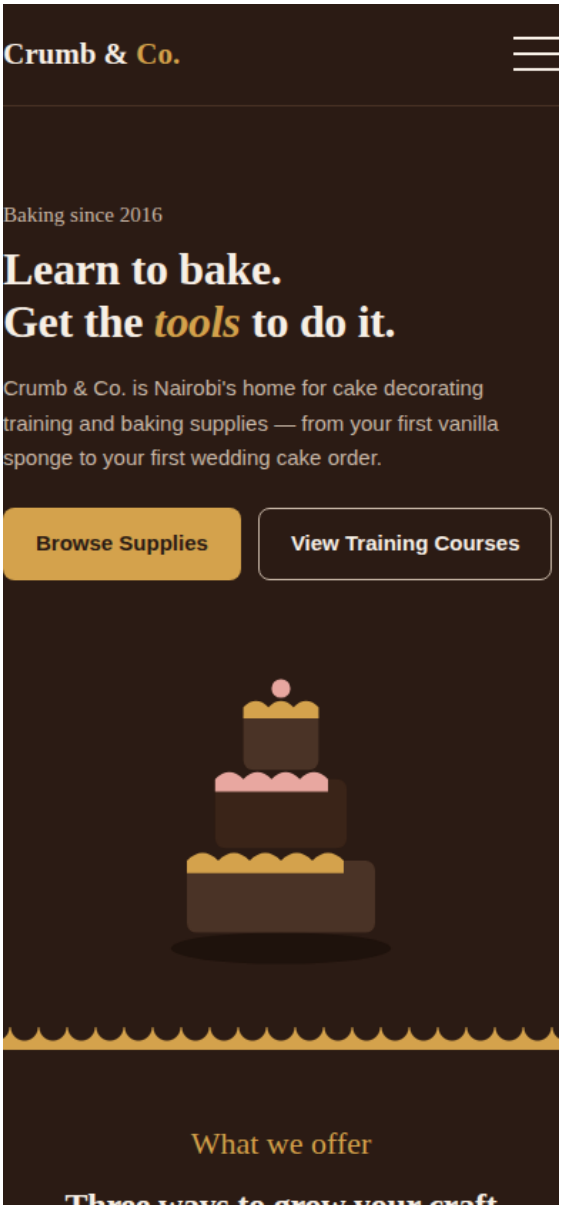


Figure 8.6: Crumb & Co. Home — mobile (390px), hamburger nav collapsed

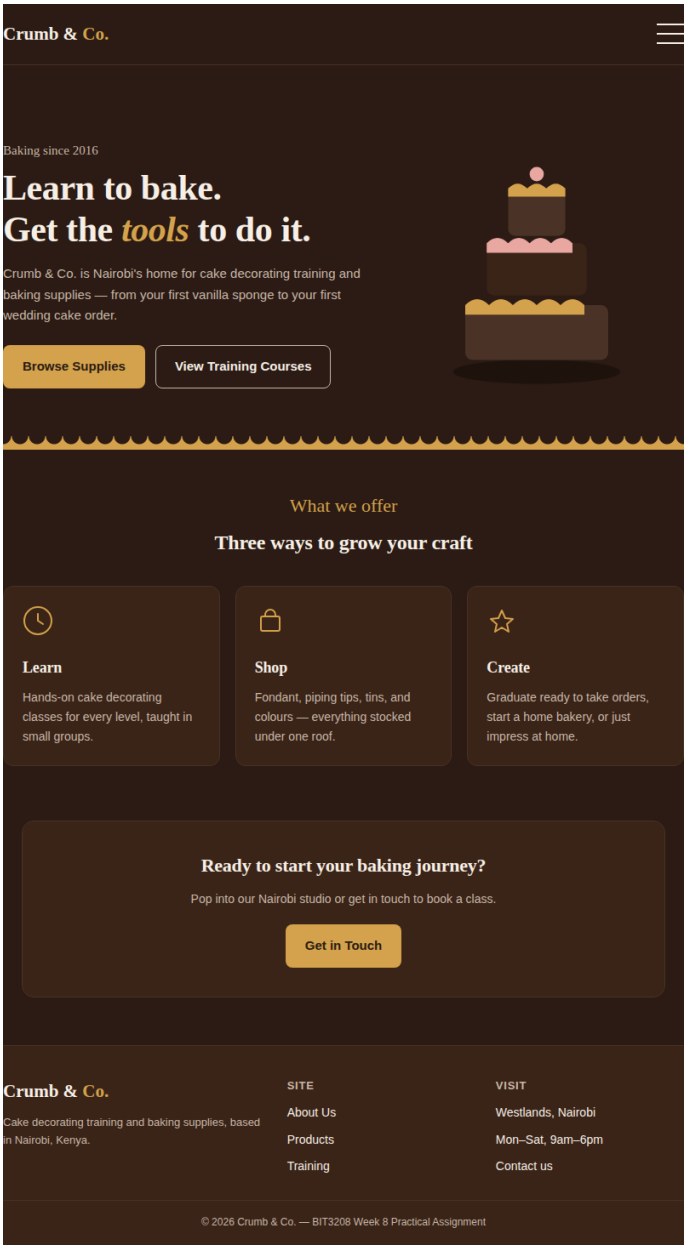


Figure 8.7: Crumb & Co. Home — tablet (800px), nav still collapsed, layout widens

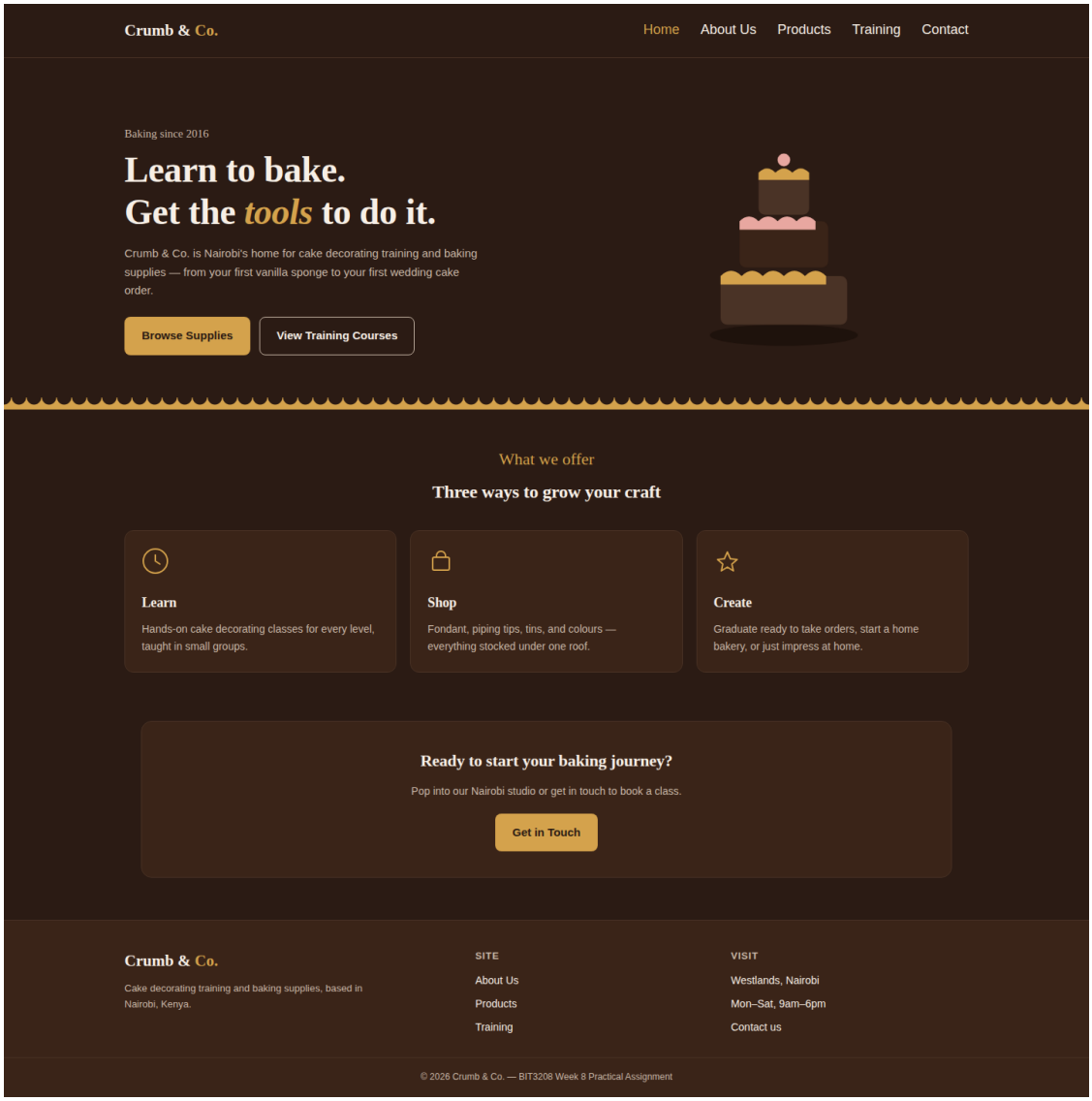


Figure 8.8: Crumb & Co. Home — desktop (1440px), horizontal nav

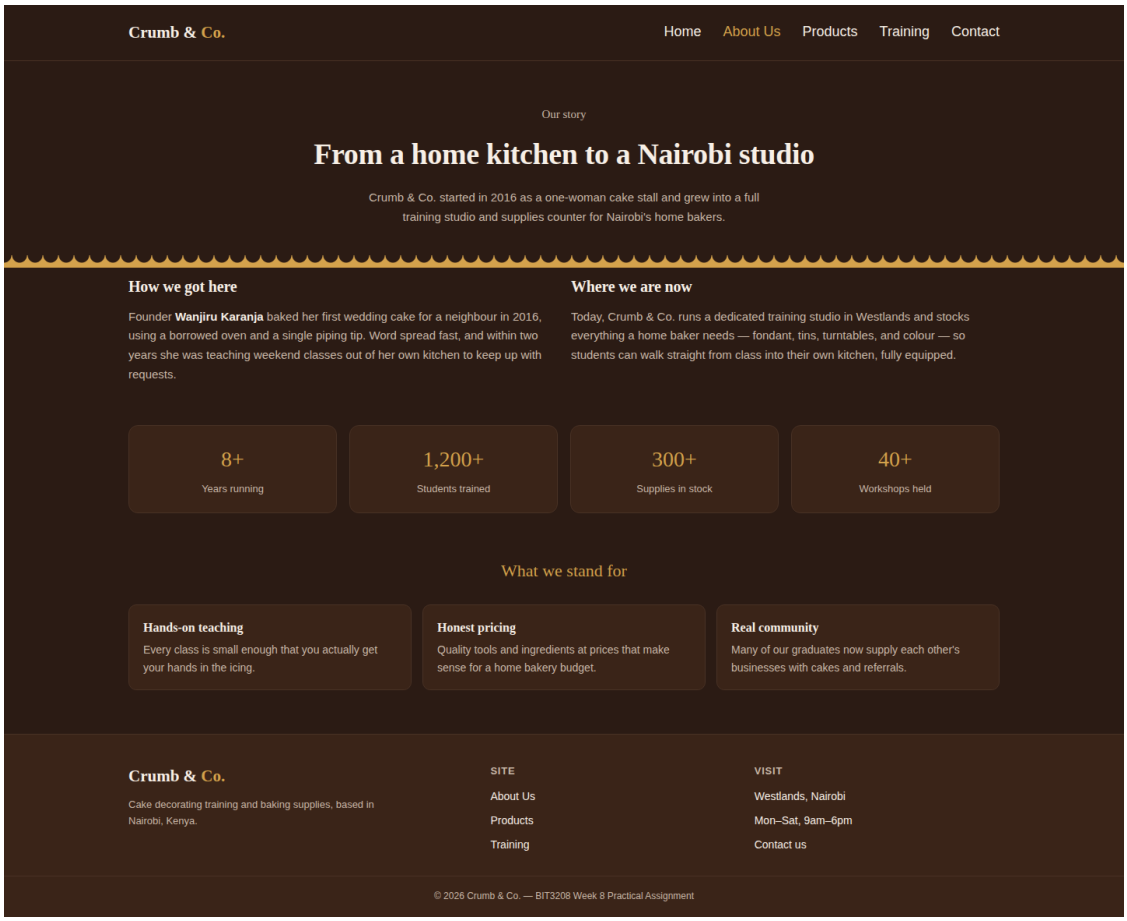


Figure 8.9: About Us page — founder story and stats grid

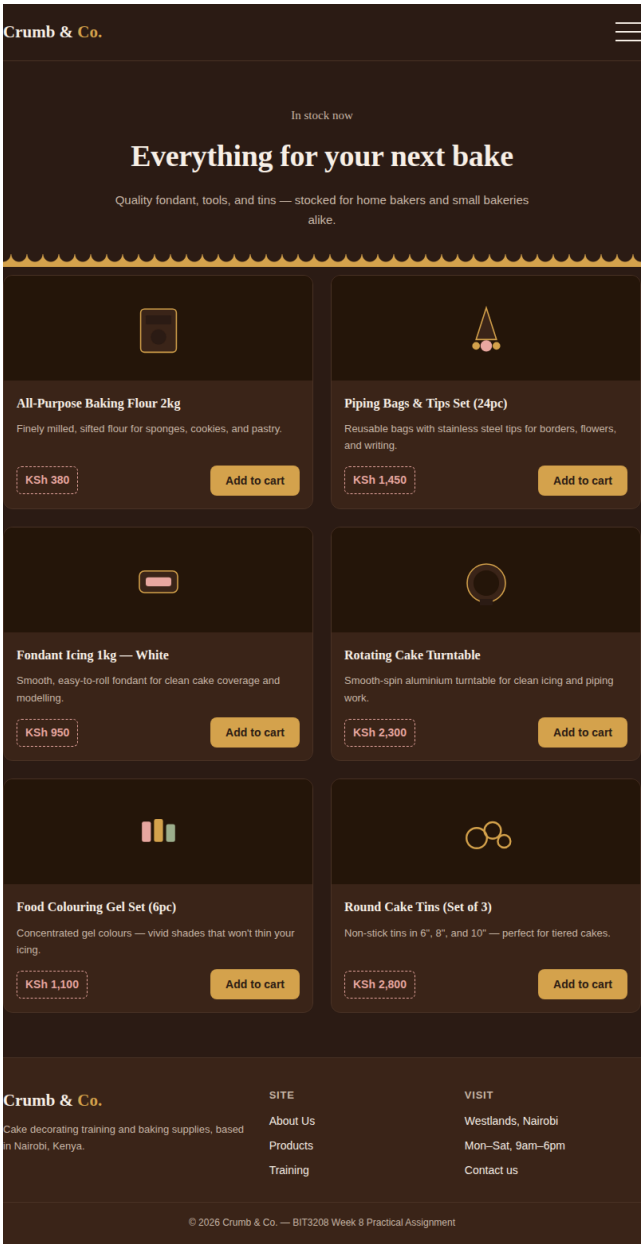


Figure 8.10: Products page — tablet (800px), grid reflows to 2 columns

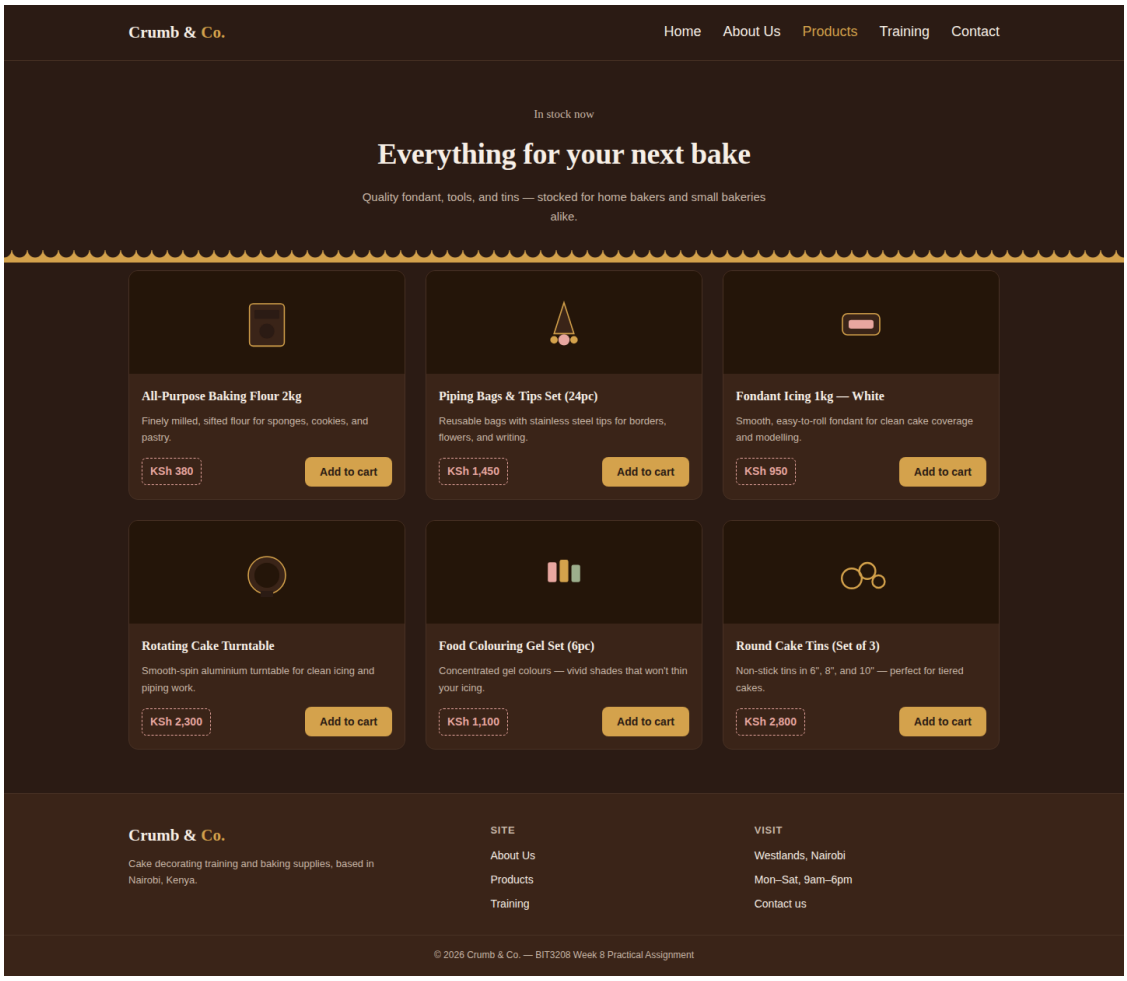


Figure 8.11: Products page — desktop (1440px), grid reflows to 3 columns

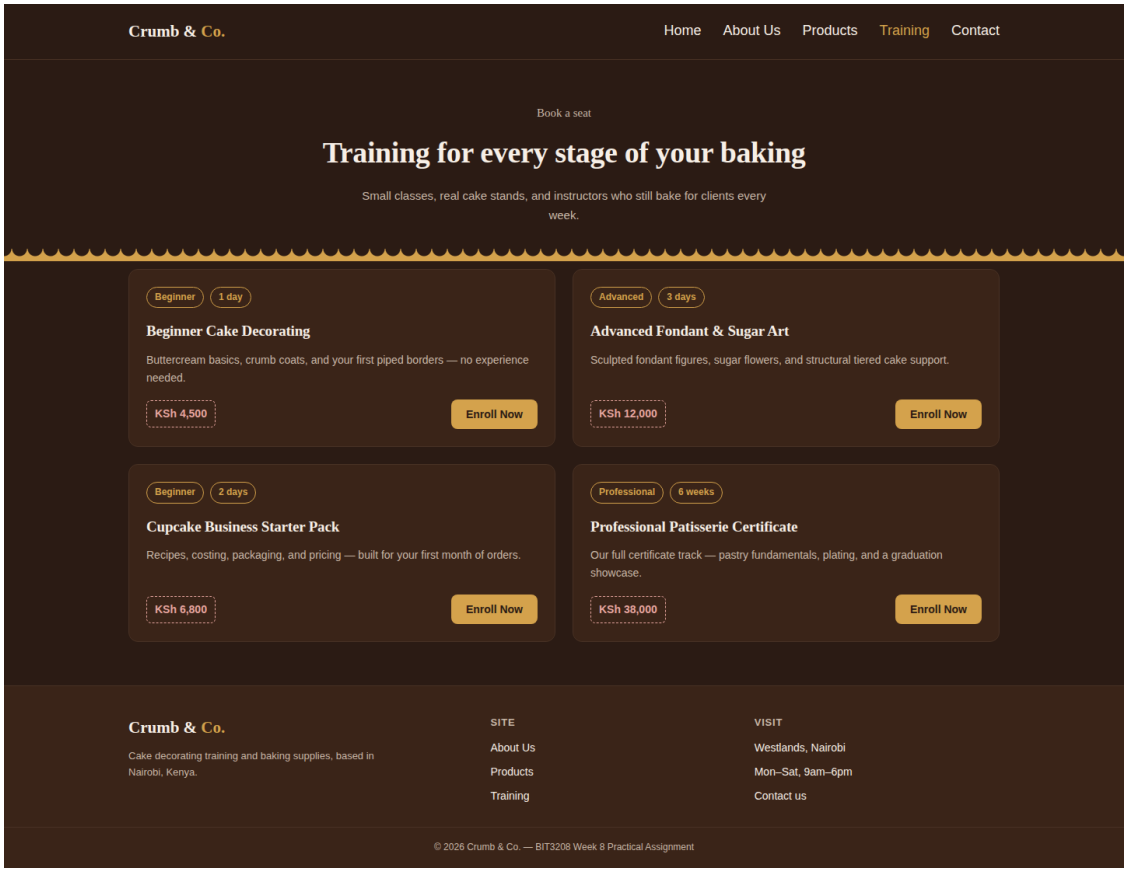


Figure 8.12: Training page — 4 course cards with duration tags

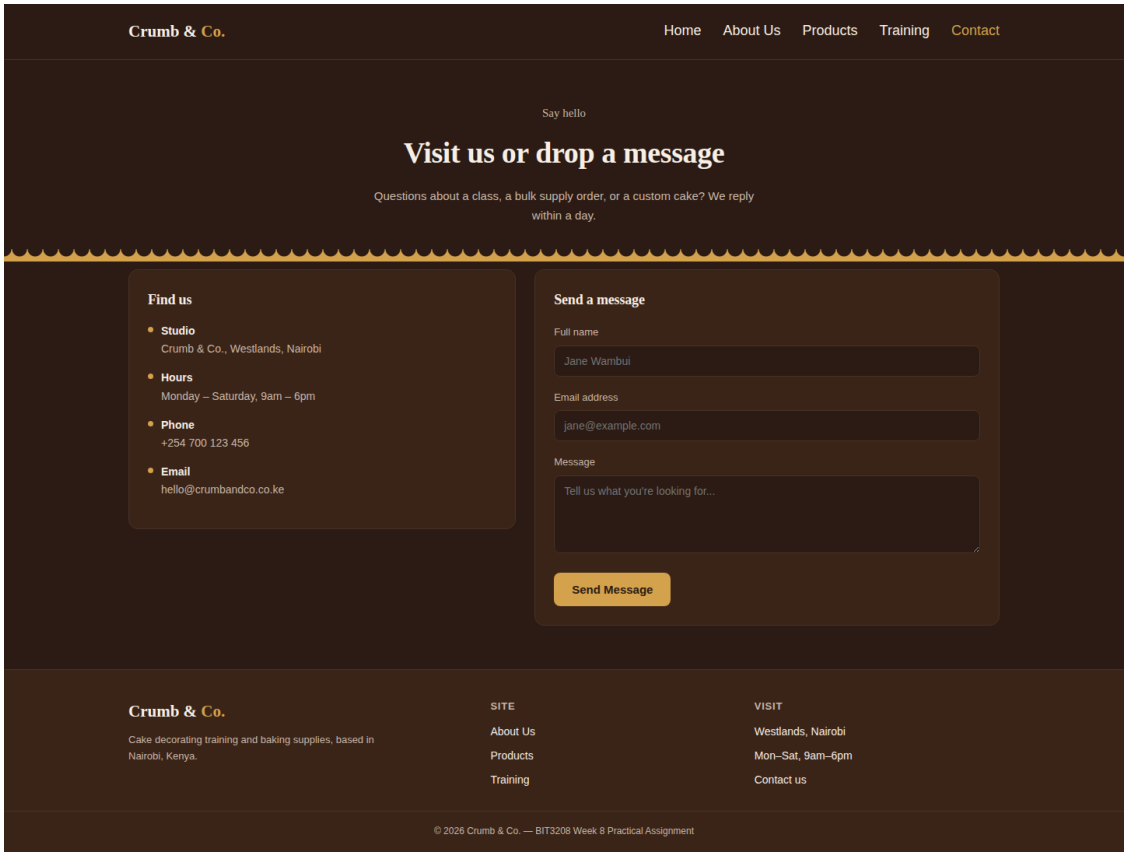


Figure 8.13: Contact page — info card beside a mobile-first contact form

Reflection

> *This was the single largest piece of work I produced this week. Duplicating the same <style> block across all five pages, rather than linking to one shared external stylesheet, introduced a meaningful amount of repeated CSS, but it kept every page genuinely self-contained — any single page can be handed in, opened, or tested on its own without depending on an external file that might not be present. If I were to rebuild this for the actual ShopMart project, I would switch to a single external stylesheet without hesitation, since maintaining six near-identical copies of the same CSS every time a single colour needs to change does not scale. Designing a second, deliberately distinct visual identity also clarified what mobile-first design actually buys you in practice: writing the single-column layout first forces an explicit decision about which content genuinely matters, before the luxury of additional columns exists to absorb anything left over.*

Challenges Faced This Week

- Getting the hamburger menu to behave correctly when the viewport crosses the desktop breakpoint while the menu is still open — resolved by applying !important to the display: flex rule inside the desktop media query, ensuring it always takes precedence over the toggled class.
- Keeping card heights consistent across the product and course grids when description text length varies between items — resolved with flex-grow: 1 on the card description element.
- Choosing breakpoints that reflect real device widths rather than arbitrary round numbers — settled on 600/700px and 900/1024px only after testing directly in Chrome DevTools' device toolbar rather than guessing.

- An unescaped & in the Google Fonts URL caused a strict HTML parser to flag an error, even though every browser renders it without issue — I corrected it regardless, since the brief explicitly requires the markup to be validated, not merely browser-compatible.

What I Learned This Week

- Mobile-first design means writing the small-screen CSS as the default case and layering additional complexity on top with min-width media queries, rather than designing for desktop and retrofitting a mobile view afterwards.
- Flexbox and Grid solve genuinely different layout problems rather than being interchangeable tools: Flexbox is suited to one-dimensional arrangements such as a navbar or hero section, while Grid is suited to two-dimensional layouts, such as a card-based showcase, where both rows and columns need to be controlled simultaneously.
- A consistent design system — a defined colour palette, typography, and one recurring signature detail — makes a multi-page site read as a single coherent product, even when it is implemented across five entirely separate HTML files with no shared backend templating.
- "It looks fine in my browser" is not equivalent to validated markup — running the actual HTML through a parser surfaced genuine errors that visual inspection alone would never have caught, since browsers are deliberately forgiving of malformed markup in ways a strict parser is not.

Week 1–8 Revision Questions

1. What is Responsive Web Design?

An approach to building a site so that its layout, images, and typography adjust automatically to whatever screen size it is viewed on, using a single codebase rather than maintaining separate mobile and desktop sites.

2. Why is Mobile-First Design important?

Because the majority of web traffic now originates from mobile devices, designing for the smallest screen first forces an explicit decision about which content and functionality genuinely matter, before the luxury of additional columns and screen real estate on larger devices is available to absorb anything left unprioritised.

3. Explain the role of media queries.

Media queries let you write CSS rules that only take effect once the viewport matches a specified condition — typically a minimum or maximum width — so that a layout can change deliberately at defined breakpoints rather than remaining fixed regardless of screen size.

4. Differentiate between Flexbox and CSS Grid.

Flexbox is designed for one-dimensional layouts — a single row or column, such as a navbar or hero section — while CSS Grid is designed for two-dimensional layouts, where rows and columns are controlled simultaneously, such as a product showcase or card grid.

5. What is the purpose of the viewport meta tag?

The viewport meta tag instructs mobile browsers to render the page at the device's actual width rather than a zoomed-out desktop-equivalent view. Without it, media queries and other responsive CSS would have effectively no visible impact on a phone, since the browser would already be simulating a much wider viewport.

6. How do responsive images improve user experience?

Responsive images scale to fit their containing element instead of overflowing the viewport or forcing horizontal scrolling, so that images display correctly and at a sensible size regardless of the device being used.

7. Explain the advantages of responsive websites.

A single codebase serves every device, which improves user experience, supports better SEO ranking (since search engines favour mobile-friendly sites), and is materially cheaper to maintain than separate mobile and desktop codebases. It also reaches a wider audience, since the site functions correctly regardless of the device a visitor happens to be using.

8. Write a CSS media query for screens smaller than 768px.

@media (max-width: 768px) { /* rules go here */ }

9. How does CSS Grid help create responsive layouts?

CSS Grid allows the number of columns to be set explicitly at each breakpoint, or determined dynamically with a function such as repeat(auto-fit, minmax(200px, 1fr)), so that the grid reflows its items to fit the available space automatically, without any manual recalculation of widths.

10. Describe two methods used to test responsive websites.

Using browser developer tools — Chrome's device toolbar, accessed via F12 — to simulate a range of screen sizes while building, and separately opening the finished site on real physical devices, such as an actual phone and tablet, to confirm it behaves consistently in practice and not only within the simulator.

Week 8 Reflection

Of everything covered in BIT3208 so far, this is the week I found most directly applicable — every site I build from this point forward, including the remainder of ShopMart, needs to work on a phone first, not as an afterthought. Building three separate responsive projects in succession — the profile page, the product showcase, and the full cake-business site — was a substantial workload for a single week, but it meant I was not simply reading about media queries in the abstract; I was actually debugging genuine breakpoint conflicts until the layout held up correctly at three distinct screen sizes rather than two.

The logical next step is to return to the earlier ShopMart weeks and verify whether those pages hold up on mobile to the same standard, since most of that earlier work has, until now, only really been tested in a desktop browser.